

DVA End-Term

Complete Study Notes

Data Visualization and Analytics

Beginner to Exam-Ready

Part 1: Google Sheets

Part 2: Python — NumPy, Pandas, Matplotlib, Seaborn

Part 3: Tableau (Theory)

+ Comprehensive Cheat Sheets

Prepared by **Parthraj Singh Bhati**

Course: Data Visualization and Analytics — End-Term

Updated: May 2026

Table of Contents

After opening this document in Microsoft Word, right-click the table below and choose “Update Field” to populate page numbers.

Table of Contents	2
PART 1 — GOOGLE SHEETS.....	7
Chapter 1 — Introduction to Data Visualization & Analytics (DVA).....	7
1.1 What is Data?	7
1.2 What is Data Analysis?.....	7
1.3 What is Data Visualization?	7
1.4 The Four Types of Analytics.....	7
1.5 The Analytics Lifecycle	8
Chapter 2 — Google Sheets Interface & Navigation.....	8
2.1 What is Google Sheets?	8
2.2 Key parts of the interface.....	8
2.3 Rows, Columns, Cells, and Ranges	9
2.4 Useful keyboard shortcuts (Windows)	9
Chapter 3 — Cell Referencing	9
3.1 Why it matters.....	9
3.2 Referring across sheets and files	10
Chapter 4 — Data Types in Google Sheets.....	10
4.1 Number formats.....	10
Chapter 5 — Sorting and Filtering.....	10
5.1 Sorting	10
5.2 Filtering.....	10
Chapter 6 — Data Cleaning and Standardization	11
6.1 Handling missing values.....	11
6.2 Find and Replace (Ctrl + H).....	11
6.3 Text to Columns	11
6.4 Remove Duplicates	11
6.5 Data Validation	11
6.6 Trim Whitespace, Clean text, Standardise case.....	12
Chapter 7 — Conditional Formatting.....	12
7.1 Setting it up.....	12
7.2 Custom-formula rules (very common in exams).....	12
7.3 Colour scales.....	12
Chapter 8 — Google Sheets Functions (Complete Reference)	14

8.1 Number Functions	14
8.2 String / Text Functions.....	15
8.3 Date & Time Functions	17
8.4 Logical Functions.....	19
8.5 Aggregate Functions	19
8.6 Conditional Aggregate Functions	20
8.7 Lookup & Reference Functions	21
8.8 Error Handling Functions.....	22
8.9 Array Functions	23
Chapter 9 — Pivot Tables and Charts	24
9.1 What is a Pivot Table?.....	24
9.2 Four pivot table areas.....	24
9.3 Creating a pivot table.....	25
9.4 Calculated fields	25
9.5 Charts	25
Chapter 10 — What-If Analysis.....	25
10.1 Goal Seek	25
10.2 Scenario Manager	26
Chapter 11 — Dashboard and Report Creation.....	26
11.1 Steps to build a dashboard in Sheets.....	26
11.2 KPI cards	26
11.3 Slicers.....	26
11.4 Dashboard design principles	27
PART 2 — PYTHON FOR DATA ANALYTICS	28
The four core libraries at a glance.....	28
Standard import block.....	28
Section A — NumPy (Numerical Python).....	29
A.1 Definition and Installation	29
A.2 Creating Arrays	29
A.3 Array Attributes	29
A.4 Indexing and Slicing	29
A.5 Boolean Masking.....	30
A.6 Fancy Indexing.....	30
A.7 Reshaping, Flattening, Concatenating, Splitting	30
A.8 Vector & Numerical Operations.....	31
A.9 Broadcasting	31

A.10 Statistical Operations	31
A.11 Matrix Operations & Linear Algebra	32
A.12 Random Module	32
Section B — Pandas (Panel Data)	34
B.1 Two core data structures	34
B.2 Loading Data	34
B.3 Inspecting Data	34
B.4 Selecting Columns and Rows	35
B.5 Data Cleaning	36
B.6 Apply, Map, Lambda	37
B.7 Sorting	37
B.8 Grouping and Aggregation	37
B.9 Pivot Tables and Crosstab	38
B.10 Merging, Joining, Concatenation	38
B.11 Descriptive Statistics & Frequency Analysis	39
B.12 Correlation Analysis	40
B.13 Outlier Detection	40
B.14 Window Functions (rolling / expanding / ewm)	40
B.15 Time Series Basics	41
B.16 Univariate, Bivariate, Multivariate Analysis	41
Section C — Matplotlib	42
C.1 Definition and Mental Model	42
C.2 Common Plot Types	42
C.3 Subplots — many charts in one figure	43
C.4 Titles, Labels, Legends, Grids, Annotations	43
C.5 Styling Tips	43
Section D — Seaborn	45
D.1 Introduction	45
D.2 Theme styling	45
D.3 Distribution Plots (univariate)	45
D.4 Relationship Plots (bivariate numeric)	45
D.5 Categorical Plots (numeric vs category)	46
D.6 Matrix / Multivariate Plots	47
D.7 Mini end-to-end example	47
PART 3 — TABLEAU (THEORY)	49
Chapter T1 — Tableau Interface	49

Chapter T2 — Tableau Data Types	49
Chapter T3 — Data Connections: Live vs Extract.....	49
Chapter T4 — Dimensions vs Measures	50
Chapter T5 — Discrete vs Continuous (Blue Pill vs Green Pill).....	50
Chapter T6 — Marks Card and Chart Building	50
Common chart types	51
Chapter T7 — Filters and Their Types.....	51
Order of operations.....	51
Filter types	51
Chapter T8 — Relationships, Joins, Union, Data Blending.....	52
Relationships (modern, default).....	52
Joins (classic)	52
Union	52
Data Blending	53
Chapter T9 — Calculated Fields and Table Calculations	53
Calculated fields	53
Table calculations	53
Chapter T10 — Level of Detail (LOD) Expressions	54
Examples	54
Chapter T11 — Sets, Parameters, Groups, Bins, Hierarchies	54
Sets	54
Parameters	54
Groups	54
Bins.....	55
Hierarchies	55
Chapter T12 — Containers and Tiles in Dashboards	55
Chapter T13 — Dashboard Design	55
Design principles	55
Chapter T14 — Storyboards	55
Chapter T15 — Advantages & Limitations of Tableau	56
Advantages.....	56
Limitations	56
FINAL CHEAT SHEETS	57
Cheat Sheet 1 — Google Sheets Functions	57
Cheat Sheet 2 — NumPy Functions	60
Cheat Sheet 3 — Pandas Functions	62

Cheat Sheet 4 — Tableau Theory	64
--------------------------------------	----

PART 1 — GOOGLE SHEETS

Goal of this part: Take you from absolute beginner in spreadsheets to confidently using Google Sheets for real data analysis — cleaning data, writing formulas, building pivots, and creating dashboards. Every concept needed for the DVA end-term is covered here.

Chapter 1 — Introduction to Data Visualization & Analytics (DVA)

1.1 What is Data?

Data is any raw fact, observation, or measurement that on its own carries no meaning. When data is organised, processed, and interpreted, it becomes information that helps people make decisions.

- **Examples of data:** 23, "Mumbai", 45.6, 2024-01-15, "Approved".
- **Examples of information:** "Average order value in Mumbai for January 2024 was ₹45.60".

1.2 What is Data Analysis?

Data analysis is the process of inspecting, cleaning, transforming, and modelling data to discover useful patterns, draw conclusions, and support decisions. It answers questions like: What happened? Why did it happen? What will happen next? What should we do about it?

1.3 What is Data Visualization?

Data visualization is the graphical representation of data — using charts, graphs, dashboards, and maps — to make patterns, trends, and outliers easier to see and faster to understand than reading raw numbers.

- Reveals trends, outliers, and relationships quickly.
- Communicates results clearly to non-technical audiences.
- Reduces cognitive load: a good chart says in 5 seconds what a table needs 5 minutes for.

1.4 The Four Types of Analytics

Analytics maturity is usually described in four stages, each answering a different question and providing more business value than the previous.

Type	Question Answered	Description	Example
Descriptive	What happened?	Summarises past data using totals, averages, counts, and charts. Looks backward.	Total sales in Q1 were ₹5 lakh; top-selling product was Shoes.
Diagnostic	Why did it happen?	Digs into causes using drill-downs, correlations, and comparisons. Asks why metrics moved.	Sales dropped in March because of a stock-out of size 9 shoes.
Predictive	What will happen?	Uses past data and statistical or ML models to forecast future outcomes.	Sales in April are expected to be ₹6 lakh based on trend and seasonality.

Type	Question Answered	Description	Example
Prescriptive	What should we do?	Recommends actions to achieve a goal, often using optimisation or simulation.	Increase ad spend on Instagram by 20% and stock size 9 shoes early to maximise April revenue.

💡 **Tip:** Memory trick: D-D-P-P → "Did, Diagnosed, Predicted, Prescribed."

1.5 The Analytics Lifecycle

Almost every analytics project follows the same six-stage cycle:

1. Define the problem — write down the business question in one clear sentence.
2. Collect data — pull from databases, surveys, files, APIs, or sensors.
3. Clean & prepare — handle missing values, fix data types, remove duplicates, standardise.
4. Analyse — explore, run statistics, group, model.
5. Visualize & communicate — build charts and dashboards for the audience.
6. Decide & act — translate findings into actions; monitor and repeat.

💡 **Tip:** In exams, you may also see this written as Problem → Data → Prepare → Model → Visualize → Deploy. The names differ; the idea is the same loop.

Chapter 2 — Google Sheets Interface & Navigation

2.1 What is Google Sheets?

Google Sheets is a free, web-based spreadsheet application by Google. It runs in the browser, autosaves to Google Drive, supports real-time collaboration, and offers most of the analysis features of Microsoft Excel plus easy sharing and add-ons.

2.2 Key parts of the interface

Element	Description
Title bar	Shows the file name; clicking it lets you rename the file.
Menu bar	File, Edit, View, Insert, Format, Data, Tools, Extensions, Help.
Toolbar	Quick buttons: undo, format, font, fill colour, borders, alignment, etc.
Name box	Shows the address of the active cell, e.g., A1. You can also type a cell or range here to jump to it.
Formula bar	Shows the actual content of the active cell (the formula, not the result).
Column headers (A, B, C...)	Letters labelling the vertical columns.
Row headers (1, 2, 3...)	Numbers labelling the horizontal rows.
Cell	Intersection of a row and column, e.g., B5. Each cell stores one value or formula.
Sheet tabs	At the bottom; each tab is a separate worksheet inside the file.
Status bar	Bottom-right; shows quick stats (sum, average, count) for any selected range.

2.3 Rows, Columns, Cells, and Ranges

- **Row:** a horizontal line of cells (e.g., row 3 = A3, B3, C3...). Maximum 10 million cells in a sheet.
- **Column:** a vertical line of cells (e.g., column B). Identified by letters: A–Z, then AA–AZ, BA–BZ...
- **Cell:** the intersection of a row and a column. Address is column letter + row number, e.g., C7.
- **Range:** a rectangular block of cells written as TopLeft:BottomRight, e.g., A1:C10 (3 columns × 10 rows = 30 cells).
- **Named range:** a friendly name for a range, set via Data → Named ranges. Use the name in formulas instead of A1:C10.

2.4 Useful keyboard shortcuts (Windows)

Shortcut	Action
Ctrl + A	Select all data in the current region.
Ctrl + Arrow	Jump to the last filled cell in that direction.
Ctrl + Shift + Arrow	Select from current cell to last filled cell.
Ctrl + ;	Insert today's date as a static value.
Ctrl + Shift + ;	Insert the current time as a static value.
Ctrl + /	Show all keyboard shortcuts.
F2	Edit the active cell.
Alt + Enter	Add a new line inside a cell.
Ctrl + Enter	Fill all selected cells with the same content.
Ctrl + D / Ctrl + R	Fill down / fill right from the cell above / to the left.

Chapter 3 — Cell Referencing

A cell reference points to another cell so a formula uses its value. Three types exist; the difference matters when you copy or drag the formula.

Type	Example	Behaviour When Copied
Relative	A1	Both column and row shift as you drag. If A1 is in B2 as =A1 and you copy it to C3, it becomes =B2.
Absolute	\$A\$1	Neither changes. \$ locks the reference. Useful for constants like tax rate.
Mixed (column locked)	\$A1	Column letter fixed; row number shifts when copied down.
Mixed (row locked)	A\$1	Row number fixed; column letter shifts when copied right.

💡 **Tip:** Press F4 while editing a reference to cycle: A1 → \$A\$1 → A\$1 → \$A1 → A1.

3.1 Why it matters

Imagine cell B1 holds GST rate 0.18. If you write `=A2*B1` in C2 and drag down, the B1 moves to B2, B3... and the formula breaks. Locking it as `=A2*$B$1` keeps every row pointing to the rate cell.

3.2 Referring across sheets and files

- **Same file, other sheet:** `=Sheet2!A1`
- **Other file:** `=IMPORTRANGE("<file-url>", "Sheet1!A1:C10")`

Chapter 4 — Data Types in Google Sheets

Sheets does not force you to declare types like a programming language, but it stores each value as one of four underlying types. Right-aligned values are usually numeric/date; left-aligned are text.

Type	Examples	Notes
Number	42, -3.14, 0.07, 1.2E5	Stored as floating-point. Currency and percentage are number formats, not separate types.
Text (string)	"Hello", "Mumbai", "007"	Anything that isn't number/date/boolean. Numbers stored as text won't add up — fix with <code>VALUE()</code> .
Boolean	TRUE, FALSE	Result of logical tests. TRUE = 1, FALSE = 0 when used in arithmetic.
Date / time	2024-08-15, 14:30:00	Internally a serial number: 1 = 30-Dec-1899; 45000 ≈ early 2023. Times are fractions of a day (0.5 = 12:00 noon).
Error	#DIV/0!, #N/A, #REF!, #VALUE!	Special values returned by failed formulas.

4.1 Number formats

Format → Number lets you change how a number is displayed without changing its underlying value. Common formats: Number, Percent, Currency, Date, Time, Scientific, Duration, Custom.

⚠ **Common Mistake:** *Formatting does not change the stored value. `=A1+1` where A1 shows "25%" actually adds 1 to 0.25, giving 1.25, not 26.*

Chapter 5 — Sorting and Filtering

5.1 Sorting

Sorting reorders rows by the values in one or more columns. Always select the entire data range (including headers) before sorting, otherwise columns can become misaligned.

7. Select the data range.
8. Data → Sort range → "Data has header row".
9. Choose column and order (A→Z or Z→A).
10. Click + Add another sort column for multi-level sort (e.g., first by Region, then by Sales descending).

⚠ **Common Mistake:** *Highlighting only one column and sorting will shuffle that column out of sync with the rest of the table — a classic data-destroying mistake.*

5.2 Filtering

Filtering hides rows that don't meet your criteria without deleting them. Two flavours:

- **Basic filter** (Data → Create a filter): adds drop-down arrows to each header. Affects everyone viewing the sheet.
- **Filter views** (Data → Create filter view): your own private filter that doesn't change what others see. Best for shared sheets.

Filter conditions include: by value (tick boxes), by condition (text contains, is between, greater than, etc.), and by colour.

Chapter 6 — Data Cleaning and Standardization

Real-world data is messy: missing values, inconsistent capitalisation, leading/trailing spaces, duplicates, mixed date formats, typos. Cleaning is usually 60–80% of an analyst's job.

6.1 Handling missing values

- **Identify blanks:** Edit → Find and replace → tick "Match entire cell" with empty Find field, or use ISBLANK() / COUNTBLANK().
- **Strategies:**
 - Delete the row if the missing field is critical and few rows are affected.
 - Fill with a constant (0, "Unknown", "N/A").
 - Fill with a summary statistic — mean/median for numbers, mode for categories.
 - Forward-fill (use the previous row's value) for time series.

6.2 Find and Replace (Ctrl + H)

Quickly fix typos and standardise values across a whole sheet.

- **Match case:** treats "Mumbai" and "mumbai" as different.
- **Match entire cell:** only replaces if the entire cell equals the search term.
- **Search using regular expressions:** use patterns like \d+ for any digits, ^Mr\. for "Mr." at the start.

6.3 Text to Columns

Splits one column into many based on a delimiter (comma, space, semicolon, custom).

11. Select the column.
12. Data → Split text to columns.
13. Choose separator: comma, semicolon, period, space, or Custom (your own character).

Example: "Singh, Parthraj, B.Tech" split by comma becomes three columns: Singh | Parthraj | B.Tech.

6.4 Remove Duplicates

14. Select the range.
15. Data → Data cleanup → Remove duplicates.
16. Pick which columns to check (all, or a subset).

 **Tip:** Keeps the first occurrence and deletes the later ones. Make a backup copy first if the data is precious.

6.5 Data Validation

Restricts what can be typed into a cell — prevents bad data from entering in the first place.

17. Select cells → Data → Data validation.
18. Choose a criteria type: List from a range, List of items, Number between, Date is valid, Text contains, Custom formula, Checkbox.
19. Choose what happens on invalid entry: "Show warning" or "Reject input".

Practical use: a Region column with a dropdown of ("North", "South", "East", "West") prevents typos like "Norht".

6.6 Trim Whitespace, Clean text, Standardise case

- **Data → Data cleanup → Trim whitespace:** removes leading, trailing, and double spaces.
- **Data → Data cleanup → Clean up suggestions:** Sheets auto-detects inconsistencies (extra spaces, mixed case) and offers one-click fixes.
- **Formulas:** =TRIM(A1), =CLEAN(A1), =UPPER(A1), =LOWER(A1), =PROPER(A1).

Chapter 7 — Conditional Formatting

Conditional formatting changes a cell's appearance based on rules — for example, paint sales below ₹10,000 red. It makes problems jump out without sorting or filtering.

7.1 Setting it up

20. Select the range.
21. Format → Conditional formatting.
22. Choose Single colour or Colour scale.
23. Pick a condition (Cell is empty, Text contains, Greater than, Date is before, Custom formula is, ...).
24. Set the formatting (fill, text colour, bold).

7.2 Custom-formula rules (very common in exams)

"Custom formula is" lets you write any logical expression returning TRUE/FALSE. The first cell in the range is the anchor — write the formula as if it applies to that cell, and Sheets adjusts the rest.

Goal	Custom formula
Highlight entire row if column D > 1000	=D2>1000
Highlight cells containing the word "urgent"	=REGEXMATCH(LOWER(A2), "urgent")
Highlight duplicates	=COUNTIF(\$A\$2:\$A\$100, A2)>1
Highlight today's date	=A2=TODAY()
Highlight weekends in a date column	=WEEKDAY(A2,2)>5

 **Tip:** Use \$ to lock the column when you want to highlight the whole row based on one column.

7.3 Colour scales

Apply a heatmap-like gradient (red → yellow → green) so smaller values appear in one colour and larger ones in another. Great for spotting hot/cold zones in sales or marks tables.

Chapter 8 — Google Sheets Functions (Complete Reference)

Functions are the heart of Sheets. Each function takes inputs (called arguments or parameters) and returns a value. Syntax: `=FUNCTION_NAME(argument1, argument2, ...)`. Required arguments appear without brackets; optional ones in square brackets in the help bubble.

 **Tip:** Always start a formula with `=`. Separate arguments with commas (in most locales) or semicolons (some European locales).

8.1 Number Functions

Used for arithmetic, rounding, random numbers, powers, and basic mathematical transformations.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
SUM	Adds all numbers in a range.	value1, [value2, ...] — numbers or ranges.	<code>=SUM(A1:A5)</code>	Sum of A1..A5
PRODUCT	Multiplies all numbers together.	value1, [value2, ...]	<code>=PRODUCT(2,3,4)</code>	24
ROUND	Rounds to a given number of decimal places.	value, places (negative places round to tens/hundreds)	<code>=ROUND(3.567, 1)</code>	3.6
ROUNDUP	Always rounds away from zero.	value, places	<code>=ROUNDUP(3.1, 0)</code>	4
ROUNDDOWN	Always rounds toward zero (truncates).	value, places	<code>=ROUNDDOWN(3.9, 0)</code>	3
MROUND	Rounds to the nearest multiple of a number.	value, factor	<code>=MROUND(17, 5)</code>	15
INT	Rounds DOWN to the nearest integer.	value	<code>=INT(7.9)</code>	7
TRUNC	Truncates digits without rounding.	value, [places]	<code>=TRUNC(7.89, 1)</code>	7.8
CEILING	Rounds UP to a multiple of significance.	value, factor	<code>=CEILING(17, 5)</code>	20
FLOOR	Rounds DOWN to a multiple of significance.	value, factor	<code>=FLOOR(17, 5)</code>	15
ABS	Returns absolute (non-negative) value.	value	<code>=ABS(-7)</code>	7
MOD	Remainder after integer division.	dividend, divisor	<code>=MOD(10, 3)</code>	1

Function	Purpose / Usage	Parameters	Example Formula	Example Output
POWER	Raises a number to a power.	base, exponent	=POWER(2, 5)	32
SQRT	Positive square root.	value	=SQRT(81)	9
EXP	e raised to a power (e $\approx$ 2.71828).	exponent	=EXP(1)	2.71828...
LN	Natural logarithm (base e).	value	=LN(EXP(2))	2
LOG	Logarithm with custom base.	value, [base=10]	=LOG(1000, 10)	3
LOG10	Logarithm base 10.	value	=LOG10(1000)	3
SIGN	Returns -1, 0, or 1 depending on sign.	value	=SIGN(-22)	-1
PI	Returns π .	(no arguments)	=PI()	3.14159...
RAND	Random decimal in [0, 1).	(no arguments — recalculates each edit)	=RAND()	0.7321... (random)
RANDBETWEEN	Random integer between two values, inclusive.	low, high	=RANDBETWEEN(1, 6)	4 (random 1..6)
RANDARRAY	Array of random decimals.	rows, cols	=RANDARRAY(2, 3)	2x3 random matrix

8.2 String / Text Functions

Used to manipulate text: extract parts, combine, change case, find substrings, replace, convert numbers/dates to formatted strings.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
LEN	Number of characters in a string.	text	=LEN("Parthraj")	8
UPPER	Converts to UPPER CASE.	text	=UPPER("hi")	HI
LOWER	Converts to lower case.	text	=LOWER("HI")	hi
PROPER	Capitalises the first letter of each word.	text	=PROPER("john doe")	John Doe
TRIM	Removes leading, trailing, and	text	=TRIM(" a b ")	"a b"

Function	Purpose / Usage	Parameters	Example Formula	Example Output
	double internal spaces.			
CLEAN	Removes non-printable characters (line breaks, tabs).	text	=CLEAN(A1)	Cleaned text
LEFT	First N characters from the left.	text, n	=LEFT("Mumbai", 3)	Mum
RIGHT	Last N characters from the right.	text, n	=RIGHT("Mumbai", 3)	bai
MID	Substring from a starting position.	text, start, length	=MID("Mumbai", 2, 3)	umb
FIND	Finds position of substring (case-sensitive).	find_text, within_text, [start]	=FIND("a", "Banana")	2
SEARCH	Same as FIND but case-INSensitive; supports wildcards.	find_text, within_text, [start]	=SEARCH("a", "Banana")	2
SUBSTITUTE	Replaces specific text with new text.	text, old, new, [occurrence]	=SUBSTITUTE("2024-05", "-", "/")	2024/05
REPLACE	Replaces N characters at a position with new text.	text, start, length, new_text	=REPLACE("ABCDE", 2, 2, "xy")	AxyDE
CONCAT	Joins two values together.	value1, value2	=CONCAT("Hello", " World")	Hello World
CONCATENATE	Joins many values into one string.	value1, value2, ...	=CONCATENATE(A1, " ", B1)	Combined string
JOIN	Joins values from an array with a delimiter.	delimiter, array	=JOIN(",", A1:A3)	a, b, c
TEXTJOIN	Like JOIN but can skip empty cells.	delim, ignore_empty, range	=TEXTJOIN(",", TRUE, A1:A5)	a,b,c
SPLIT	Splits text into separate cells by delimiter.	text, delimiter, [split_each], [remove_empty]	=SPLIT("a,b,c", ",")	Three cells: a b c
TEXT	Formats a number/date as	value, format_string	=TEXT(0.25, "0.00%")	25.00%

Function	Purpose / Usage	Parameters	Example Formula	Example Output
	text with a pattern.			
VALUE	Converts a text-numeral to a real number.	text 	=VALUE("123")	123
REPT	Repeats a string N times.	text, count	=REPT("-", 5)	-----
EXACT	TRUE if two strings match (case-sensitive).	string1, string2	=EXACT("abc", "ABC")	FALSE
CHAR	Character for a Unicode code point.	code	=CHAR(65)	A
CODE	Unicode code point of first character.	text	=CODE("A")	65
REGEXMATCH	TRUE if text matches a regex pattern.	text, regex	=REGEXMATCH("abc123", "\d+")	TRUE
REGEXEXTRACT	Returns the first regex match.	text, regex	=REGEXEXTRACT("order#123", "\d+")	123
REGEXREPLACE	Replaces all regex matches.	text, regex, replacement	=REGEXREPLACE("a1b2", "\d", "-")	a-b-

8.3 Date & Time Functions

Dates and times are stored as serial numbers. The date functions extract pieces, compute differences, or build new dates from parts.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
TODAY	Today's date (recalculates daily).	(no arguments)	=TODAY()	2026-05-18
NOW	Current date AND time.	(no arguments)	=NOW()	2026-05-18 14:32
DATE	Builds a date from year, month, day.	year, month, day	=DATE(2026, 1, 15)	2026-01-15
TIME	Builds a time from hour, minute, second.	h, m, s	=TIME(9, 30, 0)	9:30:00 AM

Function	Purpose / Usage	Parameters	Example Formula	Example Output
YEAR	Extracts year.	date	=YEAR("2026-05-18")	2026
MONTH	Extracts month number (1-12).	date	=MONTH("2026-05-18")	5
DAY	Extracts day of month (1-31).	date	=DAY("2026-05-18")	18
HOUR / MINUTE / SECOND	Extract time parts.	time	=HOUR(NOW())	14
WEEKDAY	Day of the week as number.	date, [type]	=WEEKDAY("2026-05-18", 2)	1 (Mon, type 2)
WEEKNUM	Week number of the year.	date, [type]	=WEEKNUM("2026-05-18")	21
EOMONTH	Last day of the month, N months away.	start_date, months	=EOMONTH("2026-05-18", 1)	2026-06-30
EDATE	Same day, N months away.	start_date, months	=EDATE("2026-05-18", 2)	2026-07-18
DATEDIF	Difference between dates in chosen unit.	start, end, unit ("Y"/"M"/"D"/"YM"/"YD"/"MD")	=DATEDIF("2020-01-01", TODAY(), "Y")	6 (years)
DAYS	Number of days between two dates.	end, start	=DAYS("2026-05-18", "2026-05-01")	17
NETWORKDAYS	Working days between two dates (excl. weekends).	start, end, [holidays]	=NETWORKDAYS("2026-05-01", "2026-05-18")	12
WORKDAY	Date N working days from a start date.	start, days, [holidays]	=WORKDAY(TODAY(), 5)	5 working days later
DATEVALUE	Converts text date to serial number.	date_string	=DATEVALUE("18/05/2026")	46161

Function	Purpose / Usage	Parameters	Example Formula	Example Output
TIMEVALUE	Converts text time to fraction of a day.	time_string	=TIMEVALUE("12:00")	0.5

8.4 Logical Functions

Return TRUE/FALSE or pick one value or another based on a condition.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
IF	Returns one value if test TRUE, another if FALSE.	test, value_if_true, value_if_false	=IF(A1>50, "Pass", "Fail")	Pass / Fail
IFS	Multiple conditions without nesting IFs.	cond1, val1, cond2, val2, ...	=IFS(A1>=90, "A", A1>=75, "B", A1>=50, "C", TRUE, "F")	A grade letter
SWITCH	Compare a value to many cases.	expr, case1, val1, ..., [default]	=SWITCH(A1, 1, "Mon", 2, "Tue", 3, "Wed", "?")	Day name
AND	TRUE only if every argument is TRUE.	cond1, cond2, ...	=AND(A1>0, A1<100)	TRUE / FALSE
OR	TRUE if at least one argument is TRUE.	cond1, cond2, ...	=OR(A1="N", A1="S")	TRUE / FALSE
NOT	Inverts TRUE↔FALSE.	condition	=NOT(A1>10)	FALSE if A1>10
XOR	TRUE if an odd number of conditions are TRUE.	cond1, cond2, ...	=XOR(TRUE, FALSE)	TRUE
TRUE / FALSE	Returns the boolean.	(no arguments)	=TRUE()	TRUE

💡 **Tip:** Order of cases inside IFS matters — Sheets returns the first matching one.

8.5 Aggregate Functions

Summarise ranges of values into a single number.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
COUNT	Counts numeric cells only.	value1, [value2, ...]	=COUNT(A1:A10)	Number of numbers

Function	Purpose / Usage	Parameters	Example Formula	Example Output
COUNTA	Counts non-empty cells (any type).	value1, [value2, ...]	=COUNTA(A1:A10)	Number of non-blanks
COUNTBLANK	Counts blank cells.	range	=COUNTBLANK(A1:A10)	Number of blanks
AVERAGE	Arithmetic mean of numbers.	value1, ...	=AVERAGE(A1:A10)	Mean
AVERAGEA	Mean treating text as 0 and TRUE as 1.	value1, ...	=AVERAGEA(A1:A10)	Mean incl. text
MIN / MAX	Smallest / largest value.	value1, ...	=MIN(A1:A10)	Lowest number
MEDIAN	Middle value when sorted.	value1, ...	=MEDIAN(A1:A10)	Middle value
MODE	Most frequent value.	value1, ...	=MODE(A1:A10)	Most common
STDEV / STDEVP	Sample / population standard deviation.	value1, ...	=STDEV(A1:A10)	Std dev
VAR / VARP	Sample / population variance.	value1, ...	=VAR(A1:A10)	Variance
LARGE	k-th largest value.	range, k	=LARGE(A1:A10, 2)	2nd largest
SMALL	k-th smallest value.	range, k	=SMALL(A1:A10, 2)	2nd smallest
RANK	Rank of a value in a list.	value, range, [order]	=RANK(B2, B2:B11, 0)	Position (1 = highest)
PERCENTILE	k-th percentile (0..1).	range, k	=PERCENTILE(A1:A10, 0.9)	90th percentile
QUARTILE	Quartile (0-4).	range, q	=QUARTILE(A1:A10, 3)	75th percentile
SUBTOTAL	Aggregates while respecting filters.	function_code, range	=SUBTOTAL(9, A1:A10)	Sum of visible cells

8.6 Conditional Aggregate Functions

Aggregate only the rows that meet one or more criteria. Hugely useful for slicing data without pivot tables.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
SUMIF	Sum of cells in a range that meet ONE condition.	criteria_range, criterion, [sum_range]	=SUMIF(B2:B100, "South", C2:C100)	Sales in South
SUMIFS	Sum with MULTIPLE conditions.	sum_range, range1, c1, range2, c2, ...	=SUMIFS(C2:C100, B2:B100, "South", D2:D100, ">1000")	Filtered sum

Function	Purpose / Usage	Parameters	Example Formula	Example Output
COUNTIF	Count cells meeting ONE condition.	range, criterion	=COUNTIF(B2:B100,"South")	Count of South rows
COUNTIFS	Count with MULTIPLE conditions.	range1, c1, range2, c2, ...	=COUNTIFS(B2:B100,"South", C2:C100,">1000")	Filtered count
AVERAGEIF	Mean of cells meeting ONE condition.	range, criterion, [average_range]	=AVERAGEIF(B2:B100,"South", C2:C100)	Avg South sales
AVERAGEIFS	Mean with MULTIPLE conditions.	avg_range, range1, c1, ...	=AVERAGEIFS(C2:C100, B2:B100,"South", D2:D100,">1000")	Filtered mean
MINIFS	Minimum with conditions.	min_range, range1, c1, ...	=MINIFS(C2:C100, B2:B100,"South")	Lowest South sale
MAXIFS	Maximum with conditions.	max_range, range1, c1, ...	=MAXIFS(C2:C100, B2:B100,"South")	Highest South sale
DSUM / DCOUNT / DAVERAGE	Database aggregates using a criteria table.	database, field, criteria_range	=DSUM(A1:D100, "Sales", F1:G2)	Sum filtered by criteria

 **Tip:** Criteria syntax: "South" for exact match; ">1000" for greater-than; "<>0" for not zero; "**book*" for contains "book".

8.7 Lookup & Reference Functions

Fetch a value from one table using a key found in another. The most important family of functions for data work.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
VLOOKUP	Vertical lookup: searches first column, returns value from another column.	search_key, range, index, [is_sorted=FALSE]	=VLOOKUP("P101", A2:D50, 3, FALSE)	Value from col 3
HLOOKUP	Horizontal lookup: searches first ROW.	search_key, range, row_index, [is_sorted]	=HLOOKUP("Q2", A1:E5, 3, FALSE)	Value from row 3
XLOOKUP	Modern, flexible lookup — any direction, default, exact/approx.	search_key, search_range, result_range, [if_not_found], [match_mode], [search_mode]	=XLOOKUP("P101", A2:A50, C2:C50, "Not found")	Value or "Not found"

Function	Purpose / Usage	Parameters	Example Formula	Example Output
INDEX	Returns the value at given row, col inside a range.	range, row, [column]	=INDEX(A1:D100, 5, 3)	Cell at row 5 col 3
MATCH	Position of a value within a 1-D range.	search_key, range, [match_type]	=MATCH("P101", A2:A50, 0)	Row offset
INDEX + MATCH	Combine: lookup left-to-right or any direction. More flexible than VLOOKUP.	INDEX(return_col, MATCH(key, lookup_col, 0))	=INDEX(C2:C50, MATCH("P101", A2:A50, 0))	Looked-up value
OFFSET	Cell N rows / M cols away from a reference.	reference, rows, cols, [height], [width]	=OFFSET(A1, 2, 1)	Value at B3
INDIRECT	Treats a text string as a cell reference.	ref_text, [a1_notation]	=INDIRECT("A"&ROW())	Same-row value in A
CHOOSE	Returns the Nth item from a list of arguments.	index, value1, value2, ...	=CHOOSE(2, "A", "B", "C")	B
ROW / COLUMN	Returns row / column number of a reference.	[reference]	=ROW(B5)	5
ROWS / COLUMNS	Number of rows / columns in a range.	range	=ROWS(A1:A20)	20
ADDRESS	Builds a cell address from row, column numbers.	row, col, [abs], [a1], [sheet]	=ADDRESS(2, 3)	\$C\$2
HYPERLINK	Clickable link.	url, [label]	=HYPERLINK("https://...", "Go")	Clickable "Go"
SPARKLINE	Tiny in-cell chart.	data, [options]	=SPARKLINE(A2:A10)	Mini line chart

💡 **Tip:** VLOOKUP can only look RIGHT of the key column. Use XLOOKUP or INDEX+MATCH to look left.

⚠️ **Common Mistake:** Setting `is_sorted` to `TRUE` (or omitting it in old syntax) returns the closest match; this is a frequent cause of wrong answers. Use `FALSE` for exact match.

8.8 Error Handling Functions

Errors are special values Sheets returns when a formula cannot be calculated. Knowing each error and the IFERROR/ISERROR family lets you write robust formulas.

Error	Meaning	Common cause
#DIV/0!	Division by zero.	=A1/B1 where B1 is empty or 0.
#N/A	Value not available.	VLOOKUP cannot find the key.
#NAME?	Function name unknown.	Misspelled function or unknown range name.
#REF!	Invalid reference.	Deleted cell referenced by formula.
#VALUE!	Wrong type of argument.	Adding text to a number.
#NUM!	Invalid numeric value.	SQRT(-1) or too-large value.
#NULL!	Empty intersection.	Two ranges separated by space don't intersect.
#ERROR!	Parse error.	Missing bracket or comma.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
IFERROR	Returns a fallback if formula errors.	value, [fallback]	=IFERROR(A1/B1, 0)	0 instead of #DIV/0!
IFNA	Returns a fallback ONLY for #N/A.	value, fallback	=IFNA(VLOOKUP(...), "Not found")	Friendly message
ISERROR	TRUE if value is any error.	value	=ISERROR(A1/B1)	TRUE / FALSE
ISNA	TRUE if value is #N/A.	value	=ISNA(MATCH(...))	TRUE / FALSE
ISBLANK / ISNUMBER / ISTEXT / ISLOGICAL	Type checks.	value	=ISNUMBER(A1)	TRUE / FALSE
ERROR.TYPE	Code number of the error.	value	=ERROR.TYPE(#DIV/0!)	2

8.9 Array Functions

Array formulas operate on entire ranges at once and can return many cells from a single formula. Modern Sheets often auto-expands arrays.

Function	Purpose / Usage	Parameters	Example Formula	Example Output
ARRAYFORMULA	Applies a formula to a whole range without dragging.	array_expression	=ARRAYFORMULA(A2:A100*B2:B100)	Row-wise products
FILTER	Returns rows that meet a condition.	range, cond1, [cond2, ...]	=FILTER(A2:C100, B2:B100>1000)	Rows where B>1000

Function	Purpose / Usage	Parameters	Example Formula	Example Output
SORT	Sorts a range.	range, sort_col, ascending, ...	=SORT(A2:C100, 3, FALSE)	Sorted desc by col 3
SORTN	Returns top-N rows after sorting.	range, n, [display_ties], [sort_col], [is_asc]	=SORTN(A2:C100, 5, 0, 3, FALSE)	Top-5 rows
UNIQUE	Returns distinct rows from a range.	range	=UNIQUE(A2:A100)	Unique values
QUERY	SQL-like query over a range.	data, query, [headers]	=QUERY(A1:D100, "SELECT B, SUM(D) GROUP BY B", 1)	Aggregated table
SEQUENCE	Returns a sequence of numbers.	rows, [cols], [start], [step]	=SEQUENCE(5)	1;2;3;4;5
TRANSPOSE	Swaps rows and columns.	range	=TRANSPOSE(A1:B3)	3x2 → 2x3
FLATTEN	Flattens multi-column range into one column.	range1, range2, ...	=FLATTEN(A1:B3)	Single column
SPLIT	Splits text into multiple cells.	text, delimiter, ...	=SPLIT(A1, ",")	Multi-cell output
MMULT	Matrix multiplication.	matrix_a, matrix_b	=MMULT(A1:B2, D1:E2)	Resulting matrix
SUMPRODUCT	Multiplies arrays then sums result.	array1, [array2, ...]	=SUMPRODUCT(A1:A5, B1:B5)	Weighted total

Mini examples of array formulas

Quantity in column A, price in column B. Auto-compute total in column C across 1000 rows with ONE formula in C2:

```
=ARRAYFORMULA(IF(A2:A1000="", "", A2:A1000 * B2:B1000))
```

Top-3 customers by total spend using QUERY:

```
=QUERY(A1:C, "SELECT B, SUM(C) WHERE C IS NOT NULL GROUP BY B ORDER BY SUM(C) DESC LIMIT 3 LABEL SUM(C) 'Total'", 1)
```

Chapter 9 — Pivot Tables and Charts

9.1 What is a Pivot Table?

A pivot table is an interactive summary of a large dataset. You drag fields into Rows, Columns, Values, and Filters to instantly reshape and aggregate millions of rows into a compact, readable table. It is the single most powerful built-in analysis tool in Sheets.

9.2 Four pivot table areas

Area	What goes here
Rows	Categorical field whose unique values become row labels (e.g., Region).
Columns	Categorical field whose unique values become column headers (e.g., Quarter).
Values	Numeric field to aggregate (Sum, Count, Average, Min, Max, Median, etc.).
Filters	Limits the data; values that fail the filter aren't shown.

9.3 Creating a pivot table

25. Select the data including headers.
26. Insert → Pivot table.
27. Choose location: new sheet or existing sheet.
28. On the right panel, add fields to Rows, Columns, Values, and Filters.
29. Change Summarize by (Values area) to switch aggregation.

9.4 Calculated fields

Inside a pivot table you can create custom metrics. Click Values → Add → Calculated field.
Example formula for Profit Margin: =Profit / Revenue.

9.5 Charts

Charts turn ranges (or pivot tables) into visuals. Insert → Chart, then use the chart editor on the right to pick chart type, data range, series, X-axis, and styles.

Chart type	Best for
Column / Bar	Comparing categories (sales by region).
Line	Trends over time (monthly revenue).
Area	Cumulative trends (stacked sales by product over time).
Pie / Donut	Share of a whole when categories are FEW (≤ 5).
Scatter	Relationship between two numeric variables (price vs. demand).
Histogram	Distribution of a single numeric variable (marks distribution).
Box plot (candlestick)	Spread, median, outliers.
Combo chart	Two metrics on one chart with two y-axes (revenue bars + growth-% line).
Geo chart	Values by country / state on a map.
Waterfall	Showing how a value builds up from positive and negative contributions.

 **Tip:** Always title the chart, label both axes, and remove chart-junk (3D, gradients, heavy gridlines).

Chapter 10 — What-If Analysis

What-if analysis tools answer questions like "How does the output change if I change an input?" or "What input value gives me a target output?"

10.1 Goal Seek

Goal Seek changes one input to make a formula reach a specific target. Install via Extensions → Add-ons → "Goal Seek" (official Google add-on).

Example: A loan EMI formula in cell C5 depends on principal (B1), rate (B2), tenure (B3). You want EMI = ₹10,000 — what should the tenure be?

30. Open Goal Seek.
31. Set cell: C5.
32. To value: 10000.
33. By changing cell: B3.
34. Solve. Sheets iteratively adjusts B3 until $C5 \approx 10,000$.

10.2 Scenario Manager

Scenario Manager lets you save several sets of input values and quickly switch between them to see different "what-if" outcomes. In Sheets, this is built using named ranges + a simple input-switching pattern (or via add-ons). Typical use: Best-case, Realistic, Worst-case sales forecasts.

35. Define inputs in cells (price, units, cost).
36. Create a Scenarios table listing each scenario's inputs.
37. Use a dropdown to choose the scenario; INDEX/MATCH or CHOOSE pulls the inputs.
38. Output formulas (revenue, profit) automatically update.

Chapter 11 — Dashboard and Report Creation

A dashboard is a single screen that summarises the most important metrics of a process so a manager can glance and decide. A great dashboard is glanceable, accurate, action-oriented, and audience-specific.

11.1 Steps to build a dashboard in Sheets

39. Collect & clean the raw data on a hidden "Data" sheet.
40. Build summary tables / pivots on a "Calc" sheet.
41. On the "Dashboard" sheet, place KPI cards, charts, and filters (slicers).
42. Use uniform colours, font, and spacing. Hide gridlines (View → Show → Gridlines).
43. Add a Last-updated timestamp using `=&"Updated: "&TEXT(NOW(),"dd-mmm-yyyy hh:mm")`.

11.2 KPI cards

A KPI card is just a big number with a label. Build by merging a few cells, writing the metric formula in the centre, and styling with large bold font and a subtle background.

11.3 Slicers

Slicers are interactive filter buttons attached to a chart or pivot. They let viewers filter without opening menus.

44. Data → Add a slicer.
45. Choose the data range and column to filter (e.g., Region).
46. Click the slicer and pick values to keep.
47. All charts / pivots using the same data range update instantly.

💡 **Tip:** *Slicers respect each user's view — different viewers can apply different slicer values without affecting each other.*

11.4 Dashboard design principles

- Top-down information flow: most important KPIs at the top, supporting details below.
- Limit colour: use one accent colour for highlights and grey for context.
- Round numbers (₹1.2 cr is easier than ₹12,03,847).
- Use comparisons (vs. last month, vs. target) to give numbers meaning.
- Never include more than 5–7 charts on one screen — choose wisely.

PART 2 — PYTHON FOR DATA ANALYTICS

Why Python? Python is the de-facto language of data analytics and data science. Its libraries cover every stage of the pipeline: NumPy (numerical arrays), Pandas (tabular data), Matplotlib (plots), and Seaborn (statistical visualisation). Sheets handles thousands of rows comfortably; Python easily handles millions and supports automation and reproducibility.

The four core libraries at a glance

Library	Built on	Strength
NumPy	C	Fast numerical arrays, broadcasting, math, linear algebra.
Pandas	NumPy	DataFrames for tabular data; loading, cleaning, grouping, joining.
Matplotlib	Pure Python	Low-level plotting — total control over every element.
Seaborn	Matplotlib	High-level statistical charts with great defaults.

Standard import block

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline    # for Jupyter notebooks
sns.set_theme(style="whitegrid")
```

Section A — NumPy (Numerical Python)

A.1 Definition and Installation

NumPy is the foundation library for numerical computing in Python. Its core object is the ndarray — an n-dimensional array of homogeneous data (all elements share the same type). Operations on ndarrays are vectorised, meaning they run in C-level loops and are 10–100× faster than equivalent Python lists.

```
pip install numpy
import numpy as np
```

A.2 Creating Arrays

```
# From a Python list
a = np.array([1, 2, 3, 4])          # 1-D array, shape (4,)
b = np.array([[1, 2], [3, 4], [5, 6]]) # 2-D array, shape (3, 2)

# Filled arrays
np.zeros((3, 4))      # 3x4 of zeros
np.ones((2, 3))       # 2x3 of ones
np.full((2, 2), 7)    # 2x2 of 7s
np.eye(4)             # 4x4 identity matrix
np.identity(3)        # 3x3 identity matrix (same idea)

# Ranges
np.arange(0, 10, 2)    # 0,2,4,6,8 – like range() but as ndarray
np.linspace(0, 1, 5)  # 5 evenly spaced numbers from 0 to 1 inclusive

# Random (see A.10 for the full random module)
np.random.rand(2, 3)   # uniform [0,1)
np.random.randn(2, 3)  # standard normal (mean 0, std 1)
np.random.randint(0, 10, 5) # 5 ints in [0,10)
```

A.3 Array Attributes

Attribute	Meaning
a.shape	Tuple of dimensions, e.g., (3, 2).
a.ndim	Number of dimensions (1 for 1-D, 2 for matrix, ...).
a.size	Total number of elements.
a.dtype	Data type of elements (int64, float64, bool, ...).
a.itemsize	Bytes per element.
a.nbytes	Total bytes used.
a.T	Transpose.

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape)  # (2, 3)
print(a.ndim)   # 2
print(a.size)   # 6
print(a.dtype)  # int64
```

A.4 Indexing and Slicing

NumPy indexing is very similar to Python lists but extends to multiple axes.

```
a = np.array([10, 20, 30, 40, 50])
a[0]      # 10
a[-1]     # 50
a[1:4]    # [20, 30, 40] (stop is exclusive)
a[::2]    # [10, 30, 50] (step)

m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
m[0, 2]   # 3 (row 0, col 2)
m[:, 1]   # [2, 5, 8] (all rows, col 1)
m[1, :]   # [4, 5, 6] (row 1, all cols)
m[0:2, 1:] # [[2,3], [5,6]] (top-right 2x2 sub-matrix)
```

⚠ **Common Mistake:** *Slicing returns a VIEW (not a copy). Modifying the slice changes the original. Use `.copy()` if you need an independent array.*

A.5 Boolean Masking

Use a boolean array of the same shape to pick elements where the mask is True.

```
a = np.array([5, 12, 7, 3, 19, 11])

mask = a > 10      # [False, True, False, False, True, True]
a[mask]           # [12, 19, 11]
a[a > 10]         # shorthand: [12, 19, 11]

# Compound conditions need & (and) and | (or) – never the Python keywords
a[(a > 5) & (a < 15)] # [12, 7, 11]

# Replace values matching a condition
a[a < 10] = 0      # values < 10 become 0
```

A.6 Fancy Indexing

Pass an array of indices to fetch many elements at once.

```
a = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
a[idx]      # [10, 30, 50]

m = np.array([[1,2,3],[4,5,6],[7,8,9]])
m[[0, 2], [1, 2]] # [2, 9] – picks elements (0,1) and (2,2)
```

A.7 Reshaping, Flattening, Concatenating, Splitting

```
a = np.arange(12)      # [0,1,2,...,11]
a.reshape(3, 4)        # 3x4 matrix
a.reshape(-1, 6)       # -1 = infer this dim → 2x6

m = np.array([[1,2],[3,4]])
m.flatten()            # [1,2,3,4] (returns copy)
m.ravel()              # [1,2,3,4] (returns view if possible)

# Concatenation
x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])
np.concatenate([x, y], axis=0) # vertical (4x2)
np.concatenate([x, y], axis=1) # horizontal (2x4)
```

```

np.vstack([x, y])          # vertical shortcut
np.hstack([x, y])          # horizontal shortcut
np.stack([x, y])           # adds new axis (2x2x2)

# Splitting
z = np.arange(12).reshape(3, 4)
np.split(z, 2, axis=1)     # two 3x2 chunks
np.hsplit(z, 2)            # same as above
np.vsplit(z, 3)            # three 1x4 rows

```

A.8 Vector & Numerical Operations

Arithmetic between ndarrays of the same shape happens element-wise. NumPy provides hundreds of universal functions (ufuncs).

```

a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

a + b      # [11, 22, 33, 44]
a * b      # [10, 40, 90, 160] ← element-wise, NOT dot product
a ** 2     # [1, 4, 9, 16]
np.sqrt(a) # [1, 1.41, 1.73, 2]
np.log(b)
np.exp(a)
np.sin(a)
np.abs(a - 3)

```

A.9 Broadcasting

Broadcasting is NumPy's rule for combining arrays of different shapes. The smaller array is virtually "stretched" to match the larger one — without copying memory. Two dimensions are compatible if they are equal OR one of them is 1.

```

a = np.array([[1, 2, 3],
              [4, 5, 6]])      # shape (2, 3)
b = np.array([10, 20, 30])    # shape (3,) → broadcast to (2,3)

a + b
# [[11, 22, 33],
#  [14, 25, 36]]

# Subtract the mean of every column
a - a.mean(axis=0)

```

A.10 Statistical Operations

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>np.sum(a, axis)</code>	Sum of elements along an axis.	<code>a, [axis]</code>	<code>np.sum(m, axis=0)</code>	Column sums
<code>np.mean(a, axis)</code>	Arithmetic mean.	<code>a, [axis]</code>	<code>np.mean(m)</code>	Overall mean
<code>np.median(a, axis)</code>	Median.	<code>a, [axis]</code>	<code>np.median([1,5,3])</code>	3.0

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>np.std(a, axis)</code>	Standard deviation.	<code>a, [axis]</code>	<code>np.std([1,2,3,4])</code>	1.118
<code>np.var(a, axis)</code>	Variance.	<code>a, [axis]</code>	<code>np.var([1,2,3,4])</code>	1.25
<code>np.min / np.max</code>	Min / max value.	<code>a, [axis]</code>	<code>np.max(m, axis=1)</code>	Row-wise max
<code>np.argmin / np.argmax</code>	Index of min / max.	<code>a, [axis]</code>	<code>np.argmax([7,2,9,4])</code>	2
<code>np.percentile(a, q)</code>	Percentile.	<code>a, q∈[0,100]</code>	<code>np.percentile(m, 75)</code>	Q3 value
<code>np.quantile(a, q)</code>	Quantile (q in [0,1]).	<code>a, q</code>	<code>np.quantile(m, 0.5)</code>	Median
<code>np.cumsum / np.cumprod</code>	Cumulative sum / product.	<code>a, [axis]</code>	<code>np.cumsum([1,2,3])</code>	[1,3,6]
<code>np.unique(a)</code>	Sorted distinct values.	<code>a, [return_counts]</code>	<code>np.unique([1,2,1])</code>	[1,2]
<code>np.sort(a, axis)</code>	Sorted copy.	<code>a, [axis]</code>	<code>np.sort([3,1,2])</code>	[1,2,3]
<code>np.argsort(a)</code>	Indices that would sort the array.	<code>a</code>	<code>np.argsort([3,1,2])</code>	[1,2,0]
<code>np.where(cond, x, y)</code>	Element-wise IF: x if cond else y.	condition, x, y	<code>np.where(a>0, 1, -1)</code>	Signs
<code>np.select(conds, choices)</code>	Multi-condition where.	list of conds, list of choices, default	see code	Vectorised IFS
<code>np.clip(a, lo, hi)</code>	Clip values to a range.	<code>a, low, high</code>	<code>np.clip(a, 0, 10)</code>	Bounded array

A.11 Matrix Operations & Linear Algebra

```

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A.dot(B)          # matrix product    = A @ B
np.matmul(A, B)   # same as A @ B
A @ B             # Pythonic operator
A.T               # transpose

# Linear algebra submodule
np.linalg.inv(A)   # inverse
np.linalg.det(A)   # determinant
np.linalg.eig(A)   # eigenvalues, eigenvectors
np.linalg.solve(A, b) # solve Ax = b
np.linalg.norm(v)  # Euclidean norm of a vector

```

A.12 Random Module

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>np.random.seed(s)</code>	Fixes the random seed so results are reproducible.	<code>s : int</code>	<code>np.random.seed(42)</code>	(no output)
<code>np.random.rand(d1, d2, ...)</code>	Uniform [0,1).	shape dims	<code>np.random.rand(2,2)</code>	2x2 random floats
<code>np.random.randn(d1, d2, ...)</code>	Standard Normal.	shape dims	<code>np.random.randn(3)</code>	<code>[-0.4, 1.2, 0.1]</code>
<code>np.random.randint(low, high, size)</code>	Random integers.	low, [high], [size]	<code>np.random.randint(1,7,3)</code>	<code>[2, 5, 6]</code>
<code>np.random.choice(a, size, replace, p)</code>	Pick from an array.	a, size, [replace], [p]	<code>np.random.choice([1,2,3],2)</code>	<code>[3, 1]</code>
<code>np.random.shuffle(a)</code>	Shuffles array in place.	a	<code>np.random.shuffle(a)</code>	Reordered a
<code>np.random.permutation(a)</code>	Returns shuffled copy.	a or int n	<code>np.random.permutation(5)</code>	<code>[3,0,4,1,2]</code>
<code>np.random.normal(mu, sigma, size)</code>	Normal with given mean/std.	loc, scale, size	<code>np.random.normal(0,1,3)</code>	Normal sample
<code>np.random.uniform(low, high, size)</code>	Uniform in a range.	low, high, size	<code>np.random.uniform(0,10,3)</code>	<code>[2.1, 7.4, 5.0]</code>

💡 **Tip:** Always set `np.random.seed(42)` at the top of an analysis if you want exactly the same output each run.

Section B — Pandas (Panel Data)

B.1 Two core data structures

Object	What it is
Series	A 1-D labelled array. Like one column of a spreadsheet, but with an explicit index.
DataFrame	A 2-D labelled table — rows × columns. Each column is a Series. Like a sheet.

```
import pandas as pd

# Series
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="Sales")

# DataFrame
df = pd.DataFrame({
    "Name": ["Ankit", "Bhavna", "Chirag"],
    "Marks": [88, 76, 92],
    "City": ["Delhi", "Mumbai", "Pune"],
})
```

B.2 Loading Data

Function	Purpose / Usage	Parameters	Example Formula	Example Output
pd.read_csv(path)	Read a CSV file.	path, sep, header, index_col, na_values, parse_dates, usecols, nrows	pd.read_csv('sales.csv')	DataFrame
pd.read_excel(path)	Read an Excel file.	path, sheet_name, header	pd.read_excel('a.xlsx', sheet_name='Q1')	DataFrame
pd.read_json(path)	Read JSON.	path	pd.read_json('data.json')	DataFrame
pd.read_html(url)	Scrape <table> tags.	url or html	pd.read_html('https://...')	List of DataFrames
pd.read_sql(q, conn)	Run SQL on a connection.	query, connection	pd.read_sql('SELECT * FROM t', conn)	DataFrame
df.to_csv(path)	Save as CSV.	path, index=False, sep	df.to_csv('out.csv', index=False)	Saves file
df.to_excel(path)	Save as Excel.	path, sheet_name, index	df.to_excel('out.xlsx', index=False)	Saves file

B.3 Inspecting Data

Method/Attribute	What it shows
df.head(n)	First n rows (default 5).
df.tail(n)	Last n rows.
df.sample(n)	Random n rows.
df.shape	(rows, columns) tuple.
df.size	Total cells.
df.columns	Index of column names.
df.index	Index of row labels.
df.dtypes	Type of each column.
df.info()	Concise summary: rows, columns, dtypes, non-null counts, memory.
df.describe()	Statistical summary of numeric columns (count, mean, std, min, quartiles, max).
df.describe(include='all')	Includes object columns (count, unique, top, freq).
df['col'].value_counts()	Frequency of each category in a column.
df['col'].nunique()	Number of unique values.
df.memory_usage()	Bytes used per column.

B.4 Selecting Columns and Rows

```
# Column
df["Marks"]           # Series
df[["Name", "Marks"]] # DataFrame with two columns

# Rows by position with iloc
df.iloc[0]            # first row (Series)
df.iloc[0:3]          # first three rows
df.iloc[0:3, 0:2]     # first 3 rows, first 2 columns

# Rows by LABEL with loc
df.loc[0, "Marks"]     # value
df.loc[df["Marks"] > 80] # rows where Marks > 80
df.loc[:, "Name":"Marks"] # columns from Name to Marks inclusive

# Quick boolean filter
df[df["City"] == "Pune"]

# Combined conditions
df[(df["Marks"] > 75) & (df["City"] != "Pune")]

# query() – more readable for complex conditions
df.query("Marks > 75 and City != 'Pune'")
```

 **Tip:** *.loc* is label-based (column NAMES, label INDEX). *.iloc* is integer-position-based. Mixing them is a common bug.

B.5 Data Cleaning

Missing values

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>df.isnull()</code> / <code>df.isna()</code>	DataFrame of TRUE where value is NaN.	(no args)	<code>df.isnull().sum()</code>	Missing count per column
<code>df.notnull()</code> / <code>df.notna()</code>	Opposite of isnull.	(no args)	<code>df.notnull().all()</code>	Series of booleans
<code>df.fillna(value)</code>	Replace NaN with a value or per-column mapping.	value, method, axis, inplace	<code>df.fillna({'Marks': df['Marks'].mean()})</code>	Filled DF
<code>df.fillna(method='ffill')</code>	Forward fill: use previous valid value.	method='ffill' or 'bfill'	<code>df.fillna(method='ffill')</code>	Filled DF
<code>df.dropna()</code>	Drop rows with ANY NaN.	axis, how('any'/'all'), subset, thresh	<code>df.dropna(subset=['Marks'])</code>	Cleaned DF
<code>df.interpolate()</code>	Fill NaNs using interpolation.	method='linear', etc.	<code>df.interpolate()</code>	Filled DF

Duplicates

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>df.duplicated()</code>	Boolean Series, True for duplicates after the first.	subset, keep	<code>df.duplicated().sum()</code>	Count of dupes
<code>df.drop_duplicates()</code>	Remove duplicate rows.	subset, keep('first'/'last'/'False'), inplace	<code>df.drop_duplicates(subset=['Email'])</code>	Cleaned DF

Type conversion, renaming, replacing

Function	Purpose / Usage	Parameters	Example Formula
<code>df.astype(dtype)</code>	Convert column type.	dtype or dict	<code>df['Marks'].astype(float)</code>
<code>pd.to_datetime(s)</code>	Convert text to datetime.	s, format, errors	<code>pd.to_datetime(df['Date'])</code>

Function	Purpose / Usage	Parameters	Example Formula
<code>pd.to_numeric(s)</code>	Convert to number, with <code>errors='coerce'</code> setting bad values to NaN.	<code>s</code> , <code>errors='raise'/'coerce'/'ignore'</code>	<code>pd.to_numeric(df['Qty'], errors='coerce')</code>
<code>df.rename(columns={...})</code>	Rename columns/index.	<code>columns</code> , <code>index</code> , <code>inplace</code>	<code>df.rename(columns={'Marks': 'Score'})</code>
<code>df.replace(old, new)</code>	Replace values.	<code>to_replace</code> , <code>value</code> , <code>regex</code>	<code>df['City'].replace('Dly', 'Delhi')</code>
<code>df.drop(labels, axis)</code>	Drop rows or columns.	<code>labels</code> , <code>axis(0/1)</code> , <code>inplace</code>	<code>df.drop(['City'], axis=1)</code>
<code>df.set_index(col)</code>	Set a column as the index.	<code>keys</code> , <code>inplace</code>	<code>df.set_index('ID')</code>
<code>df.reset_index()</code>	Move index back to a column.	<code>drop</code> , <code>inplace</code>	<code>df.reset_index(drop=True)</code>

B.6 Apply, Map, Lambda

```
# .apply on a Series – element-wise transformation
df["Marks"].apply(lambda x: "Pass" if x >= 40 else "Fail")

# .apply on a DataFrame with axis=1 – row-wise
df.apply(lambda row: row["Marks"] * 1.10, axis=1)

# .map on a Series with a dict – value mapping
df["Grade"] = df["Marks"].map(lambda m: "A" if m>=90 else "B" if m>=75 else "C")

# .replace with a dict – preferred for finite categories
df["City"].replace({"Bom": "Mumbai", "Dly": "Delhi"})

# .transform – same shape as input, groupwise transforms
df["Class_avg"] = df.groupby("Class")["Marks"].transform("mean")
```

B.7 Sorting

```
df.sort_values("Marks", ascending=False)
df.sort_values(["City", "Marks"], ascending=[True, False])
df.sort_index() # sort by row label
df["Marks"].rank(method="dense") # rank values (dense/min/average/first)
```

B.8 Grouping and Aggregation

`groupby()` is to pandas what pivot tables are to Sheets — split data by one or more keys, apply a function to each group, then combine.

```
# Mean marks per city
df.groupby("City")["Marks"].mean()

# Multiple aggregations at once
```

```

df.groupby("City")["Marks"].agg(["mean", "min", "max", "count"])

# Different aggregation per column
df.groupby("City").agg({"Marks": "mean", "Fees": "sum"})

# Named aggregations (clean column names)
df.groupby("City").agg(
    avg_marks=("Marks", "mean"),
    top_marks=("Marks", "max"),
    n_students=("Marks", "count"),
)

# Custom aggregation
df.groupby("City")["Marks"].agg(lambda s: s.max() - s.min())

# Multi-key grouping
df.groupby(["City", "Class"])[["Marks"]].mean()

# Iterate groups
for city, sub in df.groupby("City"):
    print(city, sub.shape)

# filter – keep entire groups meeting a condition
df.groupby("City").filter(lambda g: len(g) >= 5)

```

B.9 Pivot Tables and Crosstab

```

# pivot – simple reshape, no aggregation
df.pivot(index="Date", columns="Product", values="Sales")

# pivot_table – like Sheets' pivot table
df.pivot_table(
    index="City",
    columns="Class",
    values="Marks",
    aggfunc="mean",
    fill_value=0,
    margins=True,          # add totals row & column
)

# crosstab – frequency table of two categorical variables
pd.crosstab(df["City"], df["Class"])
pd.crosstab(df["City"], df["Class"], normalize="index") # row percentages

```

B.10 Merging, Joining, Concatenation

Function	Purpose / Usage	Parameters	Example Formula	Example Output
pd.concat([df1, df2])	Stack DataFrames vertically (default) or horizontally with axis=1.	objs, axis, ignore_index	pd.concat([a, b], axis=0)	Stacked DF

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>pd.merge(a, b, on, how)</code>	SQL-style merge on key column(s).	<code>left, right, on, how('inner'/'left'/'right'/'outer'), left_on, right_on, suffixes</code>	<code>pd.merge(orders, customers, on='cust_id', how='left')</code>	Merged DF
<code>a.join(b)</code>	Merge on the INDEX.	<code>other, how, on, lsuffix, rsuffix</code>	<code>orders.join(prices, on='product')</code>	Joined DF
<code>df.melt()</code>	Wide → long format.	<code>id_vars, value_vars, var_name, value_name</code>	<code>df.melt(id_vars='Name')</code>	Long DF
<code>df.stack()</code>	Pivot the innermost columns into row index.	(no args)	<code>df.stack()</code>	Series with MultiIndex
<code>df.unstack()</code>	Inverse of stack — innermost index level becomes columns.	<code>level</code>	<code>df.unstack()</code>	Wider DF

How joins differ

how	Behaviour
inner	Keep only keys present in BOTH tables (default).
left	Keep all rows from the left table; NaN where no match on right.
right	Mirror of left.
outer	Keep keys from either table; NaN for missing sides.
cross	Cartesian product (every row of left with every row of right).

B.11 Descriptive Statistics & Frequency Analysis

```

df.describe()                # summary for numeric cols
df.describe(include="object") # for text cols
df["Marks"].mean()
df["Marks"].median()
df["Marks"].mode()
df["Marks"].std()
df["Marks"].var()
df["Marks"].skew()           # asymmetry of distribution
df["Marks"].kurt()           # tailedness
df["Marks"].quantile([0.25, 0.5, 0.75])

# Frequency analysis
df["City"].value_counts()     # absolute counts
df["City"].value_counts(normalize=True) * 100 # percent share

# Bin a numeric column for frequency-table style analysis

```

```
pd.cut(df["Marks"], bins=[0,40,60,75,90,100], labels=["F","D","C","B","A"])

# Equal-sized buckets (quartiles)
pd.qcut(df["Marks"], q=4, labels=["Q1","Q2","Q3","Q4"])
```

B.12 Correlation Analysis

Correlation measures how strongly two variables move together, on a scale from -1 to +1. +1 = perfect positive, 0 = none, -1 = perfect negative.

```
df.corr()                # Pearson correlation matrix (numeric cols)
df.corr(method="spearman") # rank-based (handles non-linear monotonic)
df.corr(method="kendall")  # robust on small samples
df["Marks"].corr(df["Hours"]) # between two columns
sns.heatmap(df.corr(), annot=True, cmap="coolwarm") # visualise it
```

B.13 Outlier Detection

Two standard methods:

48. **IQR rule:** Any value below $Q1 - 1.5 \cdot IQR$ or above $Q3 + 1.5 \cdot IQR$ is a potential outlier (where $IQR = Q3 - Q1$).
49. **Z-score:** Any value with $|z| > 3$ (more than 3 std deviations from the mean) is a potential outlier.

```
# IQR method
Q1 = df["Sales"].quantile(0.25)
Q3 = df["Sales"].quantile(0.75)
IQR = Q3 - Q1
mask = (df["Sales"] < Q1 - 1.5*IQR) | (df["Sales"] > Q3 + 1.5*IQR)
outliers = df[mask]

# Z-score method
from scipy.stats import zscore
df["z"] = zscore(df["Sales"])
df[df["z"].abs() > 3]

# Visualise with box plot
import seaborn as sns
sns.boxplot(x=df["Sales"])
```

B.14 Window Functions (rolling / expanding / ewm)

Window functions compute statistics over a moving subset of rows. Essential for smoothing time series, computing moving averages, and running totals.

```
# 7-day moving average
df["MA7"] = df["Sales"].rolling(window=7).mean()

# 7-day moving sum, requiring at least 3 valid values
df["Sum7"] = df["Sales"].rolling(window=7, min_periods=3).sum()

# Running (expanding) total – grows each step
df["Cum"] = df["Sales"].expanding().sum()

# Exponentially weighted moving average – recent values weighted more
df["EWMA"] = df["Sales"].ewm(span=7, adjust=False).mean()
```



```
# shift / diff / pct_change for period-over-period analysis
df["Prev"] = df["Sales"].shift(1)
df["Delta"] = df["Sales"].diff()
df["Growth"] = df["Sales"].pct_change()
```

B.15 Time Series Basics

```
df["Date"] = pd.to_datetime(df["Date"])
df = df.set_index("Date").sort_index()

df.loc["2025"]           # all rows in 2025
df.loc["2025-01":"2025-03"] # Q1 2025
df.resample("M").sum()     # monthly totals
df.resample("W").mean()   # weekly means
df.asfreq("D").ffill()    # daily frequency, forward-fill gaps
```

B.16 Univariate, Bivariate, Multivariate Analysis

Type	Variables	Typical visuals
Univariate	One variable at a time.	Histogram, KDE, box plot, bar (category), value_counts.
Bivariate	Two variables together.	Scatter (num-num), box/violin (num-cat), stacked bar (cat-cat).
Multivariate	Three or more.	Pairplot, heatmap, scatter with hue/size, facet grids.

Always start with univariate exploration (one column at a time) to know each variable, then move to bivariate and finally multivariate.

Section C — Matplotlib

C.1 Definition and Mental Model

Matplotlib is Python's foundational plotting library. The two key objects are:

- **Figure:** the outer canvas / window holding everything.
- **Axes:** a single plot (chart area) inside the figure. A figure can contain many Axes (subplots).

```
import matplotlib.pyplot as plt

# Object-oriented (recommended) style
fig, ax = plt.subplots(figsize=(8, 5)) # 1 figure, 1 axes
ax.plot([1, 2, 3], [4, 5, 6])
ax.set_title("My first plot")
ax.set_xlabel("X")
ax.set_ylabel("Y")
plt.show()
```

C.2 Common Plot Types

Line plot — trends over an ordered variable

```
fig, ax = plt.subplots()
ax.plot(years, revenue, marker="o", linestyle="-", color="navy", label="Revenue")
ax.plot(years, profit, marker="s", linestyle="--", color="orange", label="Profit")
ax.set_title("Annual Revenue & Profit")
ax.set_xlabel("Year"); ax.set_ylabel("₹ Cr")
ax.legend(); ax.grid(True, alpha=0.3)
plt.show()
```

Bar plot — compare categories

```
fig, ax = plt.subplots()
ax.bar(["North", "South", "East", "West"], [120, 95, 80, 110], color="steelblue")
ax.set_title("Quarterly Sales by Region")
ax.set_ylabel("Sales (₹ lakh)")
plt.show()

# Horizontal version
ax.barh(regions, sales)
```

Histogram — distribution of a numeric variable

```
fig, ax = plt.subplots()
ax.hist(df["Marks"], bins=20, color="teal", edgecolor="white")
ax.set_title("Marks distribution"); ax.set_xlabel("Marks"); ax.set_ylabel("Frequency")
plt.show()
```

Scatter plot — relationship between two numerics

```
fig, ax = plt.subplots()
ax.scatter(df["Hours"], df["Marks"], c="purple", alpha=0.6, edgecolor="white")
ax.set_xlabel("Study hours"); ax.set_ylabel("Marks")
plt.show()
```

Box plot — five-number summary & outliers

```
fig, ax = plt.subplots()
ax.boxplot([class_a, class_b, class_c], labels=["A", "B", "C"])
ax.set_title("Marks by class")
plt.show()
```

Pie chart — share of a whole

```
fig, ax = plt.subplots()
ax.pie([45, 25, 20, 10],
      labels=["Mobiles", "Laptops", "Tablets", "Accessories"],
      autopct="%1.1f%%", startangle=90,
      colors=["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"])
ax.axis("equal")
plt.show()
```

C.3 Subplots — many charts in one figure

```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 7))

axes[0, 0].plot(x, y1); axes[0, 0].set_title("Line")
axes[0, 1].bar(cats, vals); axes[0, 1].set_title("Bar")
axes[1, 0].hist(data, bins=20); axes[1, 0].set_title("Histogram")
axes[1, 1].scatter(x, y2); axes[1, 1].set_title("Scatter")

plt.tight_layout() # prevents overlapping labels
plt.show()
```

C.4 Titles, Labels, Legends, Grids, Annotations

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>ax.set_title(s)</code>	Adds a title above the axes.	string, fontsize, color	<code>ax.set_title('Sales')</code>	Title text
<code>ax.set_xlabel / set_ylabel</code>	Axis labels.	string	<code>ax.set_xlabel('Year')</code>	Labelled axes
<code>ax.legend()</code>	Shows the legend using labels passed in plot calls.	loc, fontsize, frameon	<code>ax.legend(loc='upper left')</code>	Legend box
<code>ax.grid(True)</code>	Toggles grid lines.	True/False, axis, alpha	<code>ax.grid(True, alpha=0.3)</code>	Faint grid
<code>ax.set_xlim / set_ylim</code>	Set axis range.	(low, high)	<code>ax.set_ylim(0, 100)</code>	Range set
<code>ax.set_xticks / set_xticklabels</code>	Customise tick positions and labels.	list	<code>ax.set_xticks([1,2,3])</code>	Custom ticks
<code>ax.annotate(text, xy)</code>	Add an arrow/text annotation.	text, xy, xytext, arrowprops	<code>ax.annotate('Peak', xy=(3, 60))</code>	Annotation
<code>ax.text(x, y, s)</code>	Plain text at a point.	x, y, string	<code>ax.text(2, 50, 'note')</code>	Text
<code>plt.savefig(path)</code>	Save figure to a file.	path, dpi, bbox_inches	<code>plt.savefig('out.png', dpi=200, bbox_inches='tight')</code>	Saved file

C.5 Styling Tips

- **Use `plt.style.use("seaborn-v0_8-whitegrid")`** for a clean default style.
- Use `figsize=(8, 5)` for slide-friendly charts; `(12, 6)` for time-series.
- Always call `plt.tight_layout()` before saving — prevents cut-off labels.
- Save as `.png` at `dpi=300` for prints, `.svg` for crisp vector output.

Section D — Seaborn

D.1 Introduction

Seaborn is built on top of Matplotlib and is designed for statistical visualisation. It provides:

- Beautiful defaults — modern fonts, palettes, and grids out of the box.
- Higher-level functions for common statistical charts.
- Native integration with Pandas DataFrames (you pass column names, not arrays).

D.2 Theme styling

```
import seaborn as sns
sns.set_theme(style="whitegrid",          # darkgrid, white, dark, ticks
              palette="muted",          # deep, muted, pastel, bright, dark, colorblind
              context="notebook",       # paper, notebook, talk, poster
              font_scale=1.1)
```

D.3 Distribution Plots (univariate)

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>sns.histplot(data, x)</code>	Histogram with optional KDE.	data, x, hue, bins, kde	<code>sns.histplot(df, x='Marks', kde=True)</code>	Histogram + curve
<code>sns.kdeplot(data, x)</code>	Smooth kernel-density curve.	data, x, hue, fill, multiple	<code>sns.kdeplot(df, x='Marks', hue='Class', fill=True)</code>	Density curves
<code>sns.displot(data, x)</code>	Figure-level distribution plot, supports facets.	data, x, kind, col, row, hue	<code>sns.displot(df, x='Marks', col='Class')</code>	Faceted distribution
<code>sns.rugplot(data, x)</code>	Adds tiny ticks on the axis for each value.	data, x	<code>sns.rugplot(df, x='Marks')</code>	Rug ticks
<code>sns.ecdfplot(data, x)</code>	Empirical cumulative distribution.	data, x, hue	<code>sns.ecdfplot(df, x='Marks')</code>	Cumulative curve

```
sns.histplot(df["Marks"], kde=True, bins=20, color="steelblue")
plt.title("Marks distribution")
plt.show()
```

D.4 Relationship Plots (bivariate numeric)

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>sns.scatterplot(data, x, y)</code>	Scatter; supports hue, size, style.	data, x, y, hue, size, style	<code>sns.scatterplot(df, x='Hours', y='Marks', hue='Class')</code>	Scatter coloured by class

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>sns.lineplot(data, x, y)</code>	Line plot with optional confidence interval.	<code>data, x, y, hue, ci</code>	<code>sns.lineplot(df, x='Date', y='Sales')</code>	Line + CI band
<code>sns.relplot(data, x, y, kind)</code>	Figure-level scatter/line with facets.	<code>data, x, y, kind, col, row, hue</code>	<code>sns.relplot(df, x='Hours', y='Marks', col='City')</code>	Faceted relationship
<code>sns.regplot(data, x, y)</code>	Scatter with a regression line.	<code>data, x, y, order, robust, ci</code>	<code>sns.regplot(df, x='Hours', y='Marks')</code>	Scatter + best-fit line
<code>sns.lmplot(data, x, y)</code>	Like regplot but figure-level with facets.	<code>data, x, y, hue, col, row</code>	<code>sns.lmplot(df, x='Hours', y='Marks', hue='Class')</code>	Faceted regressions

D.5 Categorical Plots (numeric vs category)

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>sns.barplot(data, x, y)</code>	Bar of an aggregate (default mean) with CI.	<code>data, x, y, hue, estimator, ci</code>	<code>sns.barplot(df, x='Class', y='Marks')</code>	Mean marks by class
<code>sns.countplot(data, x)</code>	Counts of each category.	<code>data, x, hue, order</code>	<code>sns.countplot(df, x='City')</code>	Count of cities
<code>sns.boxplot(data, x, y)</code>	Box plot per category.	<code>data, x, y, hue</code>	<code>sns.boxplot(df, x='Class', y='Marks')</code>	Boxplots side by side
<code>sns.violinplot(data, x, y)</code>	Combines KDE and box.	<code>data, x, y, hue, inner, split</code>	<code>sns.violinplot(df, x='Class', y='Marks', split=True, hue='Gender')</code>	Violins
<code>sns.stripplot(data, x, y)</code>	Strip of dots per category.	<code>data, x, y, jitter, hue</code>	<code>sns.stripplot(df, x='Class', y='Marks', jitter=True)</code>	Jittered dots
<code>sns.swarmplot(data, x, y)</code>	Non-overlapping dots (auto-spread).	<code>data, x, y, hue</code>	<code>sns.swarmplot(df, x='Class', y='Marks')</code>	Swarm plot
<code>sns.pointplot(data, x, y)</code>	Mean as a point with CI; emphasises trends across categories.	<code>data, x, y, hue</code>	<code>sns.pointplot(df, x='Class', y='Marks', hue='Gender')</code>	Trend lines

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>sns.catplot(data, x, y, kind)</code>	Figure-level wrapper for all of the above.	data, x, y, kind, col, row, hue	<code>sns.catplot(df, x='Class', y='Marks', kind='box', col='Gender')</code>	Faceted box

D.6 Matrix / Multivariate Plots

Function	Purpose / Usage	Parameters	Example Formula	Example Output
<code>sns.heatmap(data)</code>	Coloured matrix; great for correlation matrices.	data, annot, fmt, cmap, vmin, vmax	<code>sns.heatmap(df.corr(), annot=True, cmap='coolwarm')</code>	Correlation heatmap
<code>sns.pairplot(data)</code>	Grid of scatter plots for every numeric pair, with histograms on diagonal.	data, hue, vars, diag_kind	<code>sns.pairplot(df, hue='Class')</code>	Pairwise scatter grid
<code>sns.jointplot(data, x, y)</code>	Scatter + marginal distributions.	data, x, y, kind('scatter'/'kde'/'hex'/'reg')	<code>sns.jointplot(df, x='Hours', y='Marks', kind='reg')</code>	Joint + marginals
<code>sns.clustermap(data)</code>	Heatmap with rows/columns reordered via hierarchical clustering.	data, method, metric, cmap	<code>sns.clustermap(df.corr())</code>	Clustered heatmap
<code>sns.FacetGrid(data, col, row)</code>	Manual faceting — flexibility for any plot type.	data, col, row, hue	<code>g = sns.FacetGrid(df, col='Class'); g.map(plt.hist, 'Marks')</code>	Facet grid

D.7 Mini end-to-end example

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("students.csv")
sns.set_theme(style="whitegrid")

# 1) Univariate
sns.histplot(df["Marks"], kde=True, bins=20)
plt.title("Marks Distribution"); plt.show()

# 2) Bivariate
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Class")
```

```
plt.title("Hours vs Marks"); plt.show()

# 3) Multivariate
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation matrix"); plt.show()

sns.pairplot(df, hue="Class", diag_kind="kde")
plt.show()
```


PART 3 — TABLEAU (THEORY)

What is Tableau? Tableau is a leading business-intelligence tool by Salesforce. You drag and drop fields to build interactive visualisations and dashboards. It connects to virtually every data source, handles huge volumes, and lets non-coders create world-class analytics.

Key products: Tableau Desktop (authoring), Tableau Server / Tableau Cloud (publishing & collaboration), Tableau Prep (data shaping), and Tableau Public (free, public-only).

Chapter T1 — Tableau Interface

Area	Purpose
Data pane (left)	Lists Dimensions and Measures from the connected source.
Shelves (Columns / Rows)	Drag fields here to define the structure of the view (axes / headers).
Marks card	Controls how each data point is drawn — colour, size, label, detail, tooltip, shape.
Filters shelf	Restricts what the view shows.
Pages shelf	Animates through slices of a dimension.
Show Me	Suggests appropriate chart types based on the fields you have selected.
Worksheet tabs (bottom)	Switch between worksheets, dashboards, and stories.
Status bar (bottom)	Quick stats and the number of marks displayed.

Chapter T2 — Tableau Data Types

Tableau identifies the type of each field with a small icon next to its name.

Type	Icon	Examples
Number (whole)	#	Integers like Quantity, Year.
Number (decimal)	#	Floating-point like Sales, Profit.
String	Abc	Country, Customer Name.
Boolean	T F	TRUE / FALSE flags.
Date	calendar	Order Date.
Date & Time	clock	Timestamp.
Geographic role	globe	City, State, Country — used by maps.

Chapter T3 — Data Connections: Live vs Extract

Mode	Live	Extract (.hyper)
How it works	Queries the source database directly every time.	Pulls a snapshot of the data into Tableau's super-fast in-memory engine.

Mode	Live	Extract (.hyper)
Freshness	Always up to date.	Only as fresh as the last refresh.
Speed	Depends on the source — can be slow for huge data.	Very fast (compressed columnar format).
Best for	Small data, real-time dashboards.	Big data, slow databases, offline use, Tableau Public.
File created	None.	.hyper extract file.

Chapter T4 — Dimensions vs Measures

Concept	Description
Dimension	Qualitative / categorical fields used to SLICE the data — produce headers, partitions, colours. Examples: Region, Product, Date, Customer Name.
Measure	Quantitative fields that are aggregated by default (SUM, AVG, COUNT...). Examples: Sales, Profit, Quantity.

Tableau automatically classifies fields when you first connect (numerics → measures, text/dates → dimensions). You can drag a field between sections to override.

Chapter T5 — Discrete vs Continuous (Blue Pill vs Green Pill)

Property	Discrete (Blue pill)	Continuous (Green pill)
What it produces	Headers (separate labels).	An axis (a continuous range).
Sort order	Can be reordered manually or alphabetically.	Follows numeric/temporal order on the axis.
Typical use	Categories, dates as parts (Year, Month), counts.	Numeric measures, dates as a timeline.
Conversion	Right-click → Convert to Continuous.	Right-click → Convert to Discrete.

 **Tip:** Dimensions are usually discrete (blue); measures are usually continuous (green). But you can mix and match — e.g., AGE as a discrete dimension or as a continuous measure depending on the chart.

Chapter T6 — Marks Card and Chart Building

Each row of data becomes one mark in the view. The Marks card decides what kind of mark and how it looks.

Property	Effect
Mark type	Bar, Line, Area, Square, Circle, Shape, Map, Pie, Text, Gantt, Polygon, Density.
Color	Encodes a field as colour (categorical or continuous).
Size	Encodes a field as size of each mark.
Label	Shows values directly on the chart.
Detail	Adds extra granularity without encoding it visually.

Property	Effect
Tooltip	Custom hover content.
Shape	Maps a categorical field to a shape (only with Shape mark type).

Common chart types

Chart	When to use
Bar chart	Compare a measure across categories.
Line chart	Trend of a measure over a continuous date axis.
Stacked bar	Show category composition alongside total.
Heatmap	Compare across two dimensions using colour intensity.
Highlight table	Cross-tab with conditional colouring.
Pie chart	Share-of-whole with few slices.
Scatter plot	Relationship between two measures, often coloured/sized by a dimension.
Map (filled/symbol)	Geographic data.
Bullet graph	Actual vs target metric.
Box plot	Distribution by category.
Tree map	Hierarchical share-of-whole using nested rectangles.
Histogram	Distribution of a measure (uses bins).
Gantt chart	Duration of activities along a timeline.
Dual-axis chart	Two measures with different scales on one chart (combine a bar and a line, etc.).

Chapter T7 — Filters and Their Types

Filters restrict which data appears in the view. Tableau evaluates them in a strict order, which is essential to understand for correct results.

Order of operations

50. Extract filters — applied when building/refreshing the extract.
51. Data Source filters — applied to all worksheets using the source.
52. Context filters — applied next; create a temporary table used by everything below.
53. Dimension filters — based on dimension values.
54. FIXED LOD expressions are calculated here (after Context, before Measure).
55. Measure filters — based on aggregated values.
56. INCLUDE / EXCLUDE LOD calculations and Table calculation filters — applied last.

Filter types

Type	Description
Extract filter	Cuts data when creating the .hyper extract — improves performance for huge sources.
Data source filter	Sheet-wide filter applied to every worksheet, like a 'default WHERE'.
Context filter	Promoted dimension filter; creates a temp table that other filters operate on. Use sparingly.
Dimension filter	Filters by categorical fields (e.g., Region = 'East').
Measure filter	Filters by aggregated values (e.g., SUM(Sales) > 100k).
Date filter	Discrete (specific year/quarter) or continuous (range).
Top N filter	Keep only the top/bottom N items by a chosen field.
Conditional filter	Filter via a formula (e.g., COUNT(Orders) > 5).
Quick filter	Filter widget exposed in the view for user interaction.
Cascading filter	One filter limits the options shown in another (Apply to: Selected sheets).

Chapter T8 — Relationships, Joins, Union, Data Blending

Relationships (modern, default)

Relationships were introduced in 2020.2. You define which fields match between tables, and Tableau picks the right level of aggregation and join type for each view automatically. Tables stay independent at the source — no early joining of all rows.

- Flexible: same relationship works across multiple worksheets at different granularities.
- Preserves the granularity of each table — no duplicated rows from accidental row-explosion.

Joins (classic)

Joins physically combine rows from two tables in the data source into one wide table. Four types:

Type	Keeps
Inner	Only rows where the key exists in BOTH tables.
Left	All rows from the LEFT table; NULL for missing right matches.
Right	All rows from the RIGHT table; NULL for missing left matches.
Full outer	All rows from BOTH tables; NULL where no match.

⚠ Common Mistake: Joins can multiply rows. Joining a 100-row orders table with a 50-row products table on a non-unique key can produce far more rows than you started with, inflating sums.

Union

Stacks tables with the same (or similar) columns on top of each other — like SQL UNION ALL. Useful for combining monthly sales sheets into a year file.

- **Manual union:** drag tables onto the canvas and choose Union.

- **Wildcard union:** automatically union all sheets/files matching a pattern, e.g. Sales_*.csv. Future-proof — new files join automatically.

Data Blending

Used when two tables come from DIFFERENT data sources or can't be joined cleanly. Tableau queries each source separately, then merges results on a linking field at the view level.

- Primary source contributes the rows; secondary source contributes aggregated values.
- Linking fields are auto-detected by name; can be edited via Data → Edit Blend Relationships.
- Slower and less flexible than relationships — use when relationships/joins can't be created.

Chapter T9 — Calculated Fields and Table Calculations

Calculated fields

New fields built from existing fields using Tableau's formula language. Created via Analysis → Create Calculated Field.

```
// Profit ratio
SUM([Profit]) / SUM([Sales])

// Custom category
IF [Sales] > 100000 THEN "High"
ELSEIF [Sales] > 50000 THEN "Medium"
ELSE "Low" END

// String manipulation
LEFT([Order ID], 3)
```

Table calculations

Computed AFTER the view is built — they operate on the table of marks, not the raw data. Used for running totals, ranks, moving averages, percent of total, year-over-year change.

Function	Purpose
RUNNING_SUM(expr)	Cumulative total across the partition.
WINDOW_SUM(expr, start, end)	Sum across a window of rows.
RUNNING_AVG / WINDOW_AVG	Running / windowed average.
RANK(expr) / RANK_DENSE / RANK_PERCENTILE	Rank within partition.
INDEX()	Row number within partition.
FIRST() / LAST()	Offset from start / end of partition.
LOOKUP(expr, offset)	Value N rows away — used for prev/next.
PREVIOUS_VALUE(expr)	Previous row's value — building recursive calcs.
PERCENT OF TOTAL	Each value as % of partition total.

 **Tip:** The 'Compute Using' option (Table across, Table down, Pane, Cell, Specific dimensions) controls how the calculation moves through the table.

Chapter T10 — Level of Detail (LOD) Expressions

LOD expressions let you compute a measure at a level of detail DIFFERENT from the one shown in the view. Three types:

Type	Behaviour
FIXED	Compute at exactly the listed dimensions, IGNORING the view's dimensions. Evaluated BEFORE dimension filters.
INCLUDE	Compute at the view's dimensions PLUS those listed. Useful when you need extra granularity for the calc.
EXCLUDE	Compute at the view's dimensions EXCLUDING the listed ones. Useful for higher-level aggregates while the view stays detailed.

Examples

```
// Total sales per customer, regardless of what's on the view
{ FIXED [Customer]: SUM([Sales]) }

// Average order size, treating each order as a separate row
{ INCLUDE [Order ID]: SUM([Sales]) }

// Region totals while the view shows Region + Category
{ EXCLUDE [Category]: SUM([Sales]) }

// Percent of total at customer level
SUM([Sales]) / { FIXED [Customer]: SUM([Sales]) }
```

 **Tip:** LOD precedence: Context filter → FIXED → Dimension filter → INCLUDE/EXCLUDE → Measure filter → view.

Chapter T11 — Sets, Parameters, Groups, Bins, Hierarchies

Sets

A set is a custom subset of a dimension's values — either a fixed list (Constant set) or members satisfying a condition (Dynamic / Top N / Conditional set).

- **Use case:** "Top 10 customers by sales" set that updates as data changes.
- **Set actions:** clicking marks in the view adds/removes them from the set — powerful for interactive dashboards.

Parameters

A parameter is a single user-input value (number, date, string, or boolean) that can be referenced in calculations, filters, reference lines, and sets. It is NOT tied to the data source.

- Common use: a What-if slider for discount % that drives a forecast.
- Combine with calculated fields and Show Parameter Control to expose the input.

Groups

Merge several dimension values into one. Created from the data pane or by selecting marks in the view and clicking Group.

- Example: combine "NY", "NJ", "CT" into a single "Tri-State" group.

Bins

Group a continuous measure into discrete buckets of equal width. Right-click measure → Create → Bins → choose bin size.

- Used to build histograms or to convert ages into 5-year cohorts.

Hierarchies

A drill-down structure on the Data pane — drag one dimension on top of another to create a hierarchy (e.g., Country → State → City → Postcode). Users can expand or collapse levels with + / - in the view.

Chapter T12 — Containers and Tiles in Dashboards

Concept	Description
Tiled layout	Default — items snap into a grid and resize when others change. Predictable but rigid.
Floating layout	Items have absolute positions and can overlap. Used for headers, KPIs, logos, transparent overlays.
Horizontal container	Holds objects side-by-side; resizing distributes width.
Vertical container	Holds objects stacked; resizing distributes height.
Layout container importance	Containers make dashboards responsive, easier to align, and far easier to maintain than free-floating items.

Chapter T13 — Dashboard Design

A Tableau dashboard combines multiple worksheets and visual elements (images, text, web pages) into one interactive view.

Design principles

- Start with the audience: what decision must they make?
- Place the most important KPI top-left (where eyes start).
- Use a fixed size (1366×768 for monitors, automatic for responsive) to control layout.
- Use Dashboard Actions (filter, highlight, URL, set, parameter) to make the dashboard interactive.
- Keep colour minimal — use one accent, one neutral. Reserve red for negatives.
- Avoid 3D, drop-shadows, and chartjunk.
- Always test on the target device size.

Chapter T14 — Storyboards

A Story is a sequence of worksheets or dashboards (called story points) that walk a viewer through an analytical narrative. Each story point can have its own annotation and view state.

- Used for executive presentations and analytical reports.

- Build via the New Story tab; drag worksheets/dashboards onto each point.
- Annotate each point in the caption area to explain what the viewer should notice.
- Use stories for guided analysis; use dashboards for self-service exploration.

Chapter T15 — Advantages & Limitations of Tableau

Advantages

- Drag-and-drop interface — no coding required.
- Connects to virtually every data source (databases, files, cloud apps).
- In-memory engine handles huge data fast.
- Beautiful default visuals and themes.
- Strong community — Tableau Public has thousands of free examples.

Limitations

- Expensive licensing (especially server).
- Limited statistical/ML capabilities compared to Python/R.
- Heavy data preparation is easier in Tableau Prep, SQL, or Python.
- Version-control of .twb files is awkward (binary XML).

FINAL CHEAT SHEETS

Quick-reference tables for last-minute revision. Every important function from the syllabus is here in one place.

Cheat Sheet 1 — Google Sheets Functions

Function	Purpose	Example
SUM	Add numbers in a range.	=SUM(A1:A10)
PRODUCT	Multiply numbers.	=PRODUCT(A1:A4)
AVERAGE	Mean of numbers.	=AVERAGE(A1:A10)
MIN / MAX	Smallest / largest value.	=MAX(A1:A10)
MEDIAN / MODE	Middle / most common value.	=MEDIAN(A1:A10)
STDEV / VAR	Std. dev. / variance.	=STDEV(A1:A10)
COUNT / COUNTA / COUNTBLANK	Count numbers / non-blanks / blanks.	=COUNTA(A1:A10)
ROUND / ROUNDUP / ROUNDDOWN	Rounding helpers.	=ROUND(A1, 2)
INT / MOD / ABS / POWER / SQRT	Math basics.	=POWER(2,10)
RAND / RANDBETWEEN	Random number / random integer.	=RANDBETWEEN(1,100)
LEN	Length of text.	=LEN(A1)
UPPER / LOWER / PROPER	Case conversion.	=PROPER(A1)
TRIM / CLEAN	Remove spaces / non-printable chars.	=TRIM(A1)
LEFT / RIGHT / MID	Substring functions.	=MID(A1,2,3)
FIND / SEARCH	Position of substring (case-sensitive / not).	=SEARCH("e",A1)
SUBSTITUTE / REPLACE	Replace by text / by position.	=SUBSTITUTE(A1,"-","/")
CONCAT / CONCATENATE / JOIN / SPLIT / TEXTJOIN	Combine / split strings.	=TEXTJOIN(",",TRUE,A1:A5)
TEXT / VALUE	Number↔text conversions.	=TEXT(A1,"0.0%")

Function	Purpose	Example
TODAY / NOW	Current date / date-time.	=TODAY()
DATE / YEAR / MONTH / DAY	Build / extract date parts.	=YEAR(A1)
EDATE / EOMONTH	Same-day / end-of-month shifted by N months.	=EOMONTH(A1,2)
DATEDIF	Difference between dates in chosen unit.	=DATEDIF(A1,B1,"Y")
WEEKDAY / WEEKNUM / NETWORKDAYS	Day of week / week no. / business days.	=NETWORKDAYS(A1,B1)
IF / IFS / SWITCH	Conditional logic.	=IFS(A1>=90,"A",A1>=75,"B",TRUE,"C")
AND / OR / NOT / XOR	Combine booleans.	=AND(A1>0,A1<100)
SUMIF / SUMIFS	Conditional sum.	=SUMIFS(C:C,B:B,"East",D:D,">100")
COUNTIF / COUNTIFS	Conditional count.	=COUNTIF(B:B,"East")
AVERAGEIF / AVERAGEIFS	Conditional mean.	=AVERAGEIFS(C:C,B:B,"East")
MINIFS / MAXIFS	Conditional min / max.	=MAXIFS(C:C,B:B,"East")
VLOOKUP / HLOOKUP	Vertical / horizontal lookup.	=VLOOKUP(A2, F:H, 3, FALSE)
XLOOKUP	Modern flexible lookup.	=XLOOKUP(A2, F:F, H:H, "NA")
INDEX / MATCH	Flexible lookup pair.	=INDEX(C:C, MATCH(A2, A:A, 0))
OFFSET / INDIRECT / CHOOSE	Dynamic referencing.	=INDIRECT("A"&ROW())
ROW / COLUMN / ADDRESS	Position helpers.	=ADDRESS(2,3)
IFERROR / IFNA	Replace errors with a fallback.	=IFERROR(A1/B1, 0)
ISBLANK / ISNUMBER / ISTEXT / ISERROR / ISNA	Type / error checks.	=ISNUMBER(A1)
ARRAYFORMULA	Apply a formula across a range.	=ARRAYFORMULA(A2:A*B2:B)
FILTER / SORT / SORTN / UNIQUE	Array shape helpers.	=FILTER(A:C, B:B>1000)
QUERY	SQL-like queries over ranges.	=QUERY(A:D,"SELECT B,SUM(D) GROUP BY B",1)

Function	Purpose	Example
SEQUENCE / TRANSPOSE / FLATTEN	Generate / reshape arrays.	=SEQUENCE(10)
SUMPRODUCT / MMULT	Sum of products / matrix multiplication.	=SUMPRODUCT(A1:A5,B1:B5)
HYPERLINK / SPARKLINE	Clickable links / inline mini-chart.	=SPARKLINE(A2:A10)
REGEXMATCH / REGEXEXTRACT / REGEXREPLACE	Regex helpers.	=REGEXMATCH(A1,"\\d+")
SUBTOTAL	Aggregation respecting filters.	=SUBTOTAL(9, A1:A100)

Cheat Sheet 2 — NumPy Functions

Function	Purpose	Example
<code>np.array(obj)</code>	Create an ndarray from a Python list/tuple.	<code>np.array([1,2,3])</code>
<code>np.arange(start, stop, step)</code>	Range as ndarray.	<code>np.arange(0,10,2)</code>
<code>np.linspace(start, stop, n)</code>	N evenly spaced values.	<code>np.linspace(0,1,5)</code>
<code>np.zeros</code> / <code>ones</code> / <code>full</code> / <code>eye</code>	Filled arrays / identity matrix.	<code>np.zeros((3,3))</code>
<code>np.empty(shape)</code>	Uninitialised array.	<code>np.empty((2,2))</code>
<code>a.shape</code> / <code>ndim</code> / <code>size</code> / <code>dtype</code>	Array properties.	<code>a.shape</code>
<code>a.reshape</code> / <code>a.flatten</code> / <code>a.ravel</code>	Change shape / flatten.	<code>a.reshape(3,4)</code>
<code>a.T</code> / <code>np.transpose(a)</code>	Transpose.	<code>a.T</code>
<code>np.concatenate</code> / <code>vstack</code> / <code>hstack</code> / <code>stack</code>	Combine arrays.	<code>np.vstack([x,y])</code>
<code>np.split</code> / <code>hsplit</code> / <code>vsplit</code>	Split arrays.	<code>np.hsplit(a, 2)</code>
<code>np.append</code> / <code>np.insert</code> / <code>np.delete</code>	Insert/remove elements.	<code>np.append(a, [9])</code>
<code>np.sort</code> / <code>np.argsort</code>	Sort / indices that would sort.	<code>np.argsort(a)</code>
<code>np.unique(a, return_counts)</code>	Distinct values (+ counts).	<code>np.unique(a)</code>
<code>np.where(cond, x, y)</code>	Element-wise IF.	<code>np.where(a>0, 1, -1)</code>
<code>np.select(conds, choices)</code>	Multi-condition IF.	<code>np.select([c1,c2],[v1,v2])</code>
<code>np.clip(a, lo, hi)</code>	Clamp values.	<code>np.clip(a, 0, 100)</code>
<code>np.sum</code> / <code>mean</code> / <code>median</code> / <code>std</code> / <code>var</code>	Aggregate stats (with axis).	<code>np.mean(a, axis=0)</code>
<code>np.min</code> / <code>max</code> / <code>argmin</code> / <code>argmax</code>	Min/max + indices.	<code>np.argmax(a)</code>
<code>np.percentile</code> / <code>np.quantile</code>	Percentile / quantile.	<code>np.percentile(a, 75)</code>
<code>np.cumsum</code> / <code>np.cumprod</code>	Cumulative aggregates.	<code>np.cumsum(a)</code>
<code>np.dot</code> / <code>np.matmul</code> / <code>a @ b</code>	Dot / matrix product.	<code>A @ B</code>
<code>np.linalg.inv</code> / <code>det</code> / <code>eig</code> / <code>solve</code> / <code>norm</code>	Linear algebra.	<code>np.linalg.inv(A)</code>
<code>np.random.seed(s)</code>	Set seed for reproducibility.	<code>np.random.seed(42)</code>
<code>np.random.rand</code> / <code>randn</code>	Uniform / normal random arrays.	<code>np.random.rand(3,3)</code>
<code>np.random.randint(low,high,size)</code>	Random integers.	<code>np.random.randint(0,10,5)</code>
<code>np.random.choice(a, size, p)</code>	Sample from an array.	<code>np.random.choice([1,2,3],2)</code>
<code>np.random.shuffle</code> / <code>permutation</code>	Shuffle in place / copy.	<code>np.random.shuffle(a)</code>

Function	Purpose	Example
np.random.normal / uniform	Normal / uniform distributions.	np.random.normal(0,1,5)
np.sqrt / log / exp / abs / power	Element-wise math.	np.sqrt(a)
np.isnan / isfinite / isinf	NaN / finite checks.	np.isnan(a)
np.nansum / nanmean / nanstd	Stats ignoring NaN.	np.nanmean(a)
np.broadcast_to / broadcast_arrays	Manual broadcasting.	np.broadcast_to(b,(3,3))

Cheat Sheet 3 — Pandas Functions

Function / Method	Purpose	Example
pd.read_csv / read_excel / read_json	Load data.	pd.read_csv('a.csv')
df.to_csv / to_excel	Save data.	df.to_csv('b.csv', index=False)
df.head / tail / sample	Peek at rows.	df.head(10)
df.info / describe / shape / dtypes	Inspect data.	df.info()
df.columns / df.index	Column / row labels.	df.columns
df['col'] / df[['c1','c2']]	Select columns.	df['Sales']
df.loc[r,c] / df.iloc[r,c]	Label / position indexing.	df.loc[0,'Name']
df.query('A>5 and B==1')	SQL-style row filter.	df.query('Marks>80')
df.isnull / notnull / isna / notna	Missing value tests.	df.isnull().sum()
df.fillna(value) / dropna()	Handle missing.	df.dropna()
df.duplicated / drop_duplicates	Find / remove dupes.	df.drop_duplicates()
df.astype / pd.to_datetime / pd.to_numeric	Type conversion.	df.astype({'A':int})
df.rename(columns=...)	Rename labels.	df.rename(columns={'a':'A'})
df.replace(old, new)	Replace values.	df.replace(0, np.nan)
df.drop(labels, axis)	Remove rows/columns.	df.drop('X', axis=1)
df.set_index / reset_index	Index management.	df.set_index('ID')
df.apply / map / transform	Apply functions.	df['A'].apply(lambda x: x*2)
df.sort_values / sort_index	Sort rows.	df.sort_values('Sales', ascending=False)
df['col'].value_counts / nunique	Frequency / uniques.	df['City'].value_counts()
df.groupby(key).agg(...)	Split-apply-combine.	df.groupby('City').agg(mean_m=('M','mean'))
df.groupby(key).transform / filter	Group-wise transforms / filtered groups.	df.groupby('C')['M'].transform('mean')
df.pivot / pivot_table	Reshape / aggregate.	df.pivot_table(index='C', columns='Q', values='S', aggfunc='sum')
pd.crosstab(a, b)	Frequency table.	pd.crosstab(df.A, df.B)
pd.merge(a, b, on, how)	SQL-style join.	pd.merge(o, c, on='id', how='left')

Function / Method	Purpose	Example
<code>df.join(other)</code>	Join on index.	<code>a.join(b)</code>
<code>pd.concat([a,b], axis)</code>	Stack DataFrames.	<code>pd.concat([a,b], ignore_index=True)</code>
<code>df.melt / stack / unstack</code>	Wide↔long reshapes.	<code>df.melt(id_vars='Name')</code>
<code>df.corr / .cov</code>	Correlation / covariance.	<code>df.corr()</code>
<code>df.rolling(window) / expanding / ewm</code>	Window functions.	<code>df['x'].rolling(7).mean()</code>
<code>df.shift / diff / pct_change / rank</code>	Period-over-period helpers.	<code>df['x'].pct_change()</code>
<code>pd.cut / pd.qcut</code>	Bin numeric → categories.	<code>pd.cut(df['M'], bins=[0,50,75,100])</code>
<code>pd.get_dummies(df)</code>	One-hot encoding.	<code>pd.get_dummies(df['City'])</code>
<code>df.resample(freq).agg(...)</code>	Time-series resampling.	<code>df.resample('M').sum()</code>

Cheat Sheet 4 — Tableau Theory

Concept	Key idea
Data types	Number (whole/decimal), String, Boolean, Date, Date-Time, Geographic.
Live vs Extract	Live = always fresh, queries source. Extract = fast snapshot in .hyper file.
Dimension vs Measure	Dimension = qualitative slicer (rows/colours). Measure = quantitative; aggregated.
Blue pill (discrete)	Produces headers; categorical or date parts.
Green pill (continuous)	Produces an axis; numeric or date timeline.
Marks card	Mark type + Color, Size, Label, Detail, Tooltip, Shape encodings.
Show Me	Suggests appropriate chart types based on selected fields.
Filter order	Extract → Data Source → Context → Dimension → FIXED LOD → Measure → INCLUDE/EXCLUDE LOD.
Filter types	Extract, Data Source, Context, Dimension, Measure, Date, Top N, Conditional, Quick, Cascading.
Relationships	Modern way to combine tables; granularity preserved; context-aware joins per view.
Joins	Inner, Left, Right, Full Outer — physically combine rows; can multiply data.
Union	Stack tables with same columns; manual or wildcard.
Data blending	Used across different sources; primary + secondary linked on view fields; happens after the query.
Calculated fields	User-defined fields using formulas (IF, CASE, arithmetic, string).
Table calculations	Post-aggregation: running sum, rank, % of total, moving averages. Direction set by Compute Using.
LOD - FIXED	Compute at listed dims, ignoring view. Evaluated before dimension filters.
LOD - INCLUDE	View dims PLUS listed ones. For extra granularity inside the calc.
LOD - EXCLUDE	View dims MINUS listed ones. For higher-level aggregates in a detailed view.
Sets	Custom subset of a dimension; constant or dynamic; usable in calcs and filters; supports set actions.
Parameters	Single user-input value — number, date, string, bool. Drive calcs/filters/ref lines.
Groups	Combine dimension members into one label.
Bins	Bucket a continuous measure into equal-width intervals (used for histograms).
Hierarchies	Drill-down structure of dimensions (Country→State→City).
Containers	Horizontal / Vertical layout containers — keep dashboards aligned and responsive.

Concept	Key idea
Tiled vs Floating	Tiled = grid-snapped. Floating = absolute position; used for overlays/KPI cards.
Dashboard	Combination of worksheets + objects with interactive actions and a fixed size for predictability.
Story	Sequence of story points telling an analytical narrative; each point has a caption.
Dashboard actions	Filter, Highlight, URL, Go to sheet, Parameter, Set — drive interactivity.